Tabular and Deep Reinforcement Learning for Optimal Multiple Stopping Problems

Abstract

Optimal multiple stopping problems with applications in finance, operations research, and statistics, involve sequential decision making about when to exercise a finite number of stopping rights under uncertainty. This paper presents a comprehensive theoretical and methodological overview of these frameworks. We formalize the multiple stopping problem as an augmented-state Markov Decision Process that explicitly tracks the remaining exercise rights, thereby enabling any general reinforcement learning method to be applied while also revealing key structural properties like threshold-type optimal policies. We extend the standard Longstaff-Schwartz Method to multiple stops, and introduce Bootstrapped Least-Squares Monte Carlo (TD-LSM), a novel temporal-difference variant of the classical Longstaff–Schwartz Method that replaces high-variance Monte Carlo realized returns with one-step bootstrapped continuation values. We empirically evaluate TD-LSM alongside tabular and deep RL baselines on simulated American option benchmarks in low-data regimes, where it consistently outperforms the classical Longstaff-Schwartz Method. We further scale these algorithms by integrating deep neural network architectures, and establish performance bounds for LSPI in our multiple stopping framework.

Keywords: Optimal Multiple Stopping, Reinforcement Learning, American Put Options, Longstaff Schwartz

1. Introduction

Optimal stopping problems study sequential decision-making under uncertainty, where an agent observes the evolution of a stochastic process and must decide when to terminate it in order to maximize an expected reward. At each time step, the agent faces a binary decision: continue observing the process and retain the option to stop later, or exercise a stopping action that yields an immediate terminal payoff. This trade-off between information acquisition and commitment lies at the core of optimal stopping theory and gives rise to rich structural and analytical properties.

Optimal stopping problems arise naturally across a wide range of scientific and engineering disciplines. In finance, classical examples include the optimal exercise of American-style options [28, 18], and asset-selling problems [20], where prices evolve stochastically and exercising too early or too late can significantly reduce value. In economics and operations research, optimal stopping underpins the house-selling problem [32], job search and secretary problems [8, 6, 36, 35], and inventory theory [32]. In statistics, stopping rules appear in sequential hypothesis testing and change-point detection [33, 31, 37], where decisions determine when to stop collecting data. More broadly, optimal stopping captures any decision problem involving an irreversible action taken under uncertainty with the option to delay.

Optimal multiple stopping = generalizes this framework by allowing the agent to exercise a fixed number $K > 1$ of stopping rights over the horizon. Rather than deciding when to stop once, the agent must determine a sequence of stopping times, subject to temporal ordering constraints. This extension substantially increases modeling expressiveness and captures decision problems involving repeated irreversible actions. Prominent examples include swing options in energy markets, where the holder may exercise multiple times [38]; repeated selling or liquidation problems, where assets are disposed of in several tranches [11]; buy–sell trading strategies, where entry and exit must both be optimally timed [23]; inventory replenishment and clearance decisions; and adaptive capacity expansion and contraction in operations. For a comprehensive overview of these diverse applications and an extensive bibliography of the underlying theory, we refer the reader to the recent monograph by Sofronov and Szajowski [34]. In these settings, exercising early alters the future opportunity set by reducing the remaining number of stopping rights, creating strong coupling between decisions across time.

If the Markovian dynamics of the underlying process are known, classical dynamic programming or methods such as value iteration or policy iteration can be used to obtain the optimal policy. However, when the environment dynamics are unknown and the agent must learn directly from observed trajectories, one must rely on reinforcement learning (RL). While a rich body of literature has successfully applied RL to single optimal stopping problems [27, 28, 30, 3], and theoretical works have formalized optimal multiple stopping as an MDP [5, 11, 34], to the best of our knowledge, we are the first to explicitly develop and evaluate RL algorithms dedicated to the optimal multiple stopping framework. Reinforcement learning (RL), specifically offline RL, offers a scalable alternative by learning from fixed datasets, a necessity in domains like quantitative finance where online interaction is costly. However, standard RL algorithms typically ignore the temporal structure of multiple stopping, leading to high variance and poor data efficiency.

To tackle this problem, we model multiple stopping as an augmented-state MDP that explicitly tracks remaining exercise rights. Our contributions are threefold:

- **Structured Formulation:** We formalize repeated–sell and buy–sell problems as MDPs, solving discrete versions to characterize their fundamental structural properties.
- **Empirical Analysis & TDLSM:** We conduct extensive benchmarking across both tabular and function-approximation settings, evaluating Least-Squares Monte Carlo (LSM), Fitted Q-Iteration (FQI), and Least-Squares Policy Iteration (LSPI). To address the high-variance limitations of standard Monte Carlo approaches, we propose TD-LSM, a novel data-efficient algorithm that combines low variance with improved sample efficiency while maintaining strong empirical performance. Across all experimental configurations including varying horizons, numbers of exercise opportunities, and dimensions of the underlying asset space, TD-LSM consistently outperforms standard LSM, and in several settings also surpasses FQI and LSPI. We further extend our empirical study to deep reinforcement learning architectures, benchmarking Deep LSM, Deep TD-LSM, Conservative Q-Learning (CQL), and Deep Q-Networks (DQN).
- **Theoretical Analysis:** We provide convergence analysis and formal convergence proofs for LSPI when adapted to the optimal multiple stopping framework.

2. Background and Related Work

Many canonical single-stop problems admit strong structural properties. In settings such as the house-selling problem [32], asset-selling problem [20], and secretary problem [7, 35], the optimal policy is often of threshold type, arising from monotonicity or convexity of the value function in time or state [1]. Foundational mathematical treatments of these sequential stopping rules and probability models are extensively detailed in classical texts [32, 6, 22, 33]. These problems can be formulated as finite-horizon Markov decision processes (MDPs) with two actions, *continue* or *stop*, consistent with the framework introduced in Section 3. Such optimal stopping frameworks form the theoretical bedrock for statistical applications including sequential hypothesis testing, quickest change-point detection [31, 37], and adaptive experimental design in clinical trials.

Multiple stopping problems extend this framework by allowing a fixed number of stopping actions. Early foundational work includes Häggström [8] and Stadje [24], who studied optimal procedures when more than one stop is permitted under IID assumptions. Kobylanski et al. [11] proved existence and uniqueness results for the value function and showed that a k-stop problem can be reduced recursively to a sequence of single-stop problems with decreasing stopping counts. In continuous time, Carmona and Touzi [5] developed a dual martingale formulation for multiple stopping, with applications to swing option valuation in energy markets.

A particularly important special case is the *double stopping* or buy-sell problem, where the agent selects one stopping time to buy and a later stopping time to sell. Sofronov [23] derived the optimal finite-horizon double stopping rule and showed that the optimal policy is characterized by two time-dependent thresholds, with the hysteresis band between thresholds defining the minimum price spread required to justify entry

and exit. Related work has studied multiple selling and repeated liquidation problems under various price dynamics and dependence structures, further confirming the emergence of threshold-type strategies [20].

When the underlying stochastic process is Markovian, optimal stopping can be cast as an MDP with a binary action space, enabling reinforcement learning approaches. Tsitsiklis and Van Roy [27] developed a function approximation framework for optimal stopping and proved convergence results for value-based methods using projection arguments in weighted L^2 spaces. Yu and Bertsekas [30] proposed least-squares temporal-difference algorithms specifically tailored to stopping problems. In finite-state and finite-horizon settings, standard tabular Q-learning [29] converges to the optimal Q-function, and backward induction or finite-horizon temporal-difference updates can be used to accelerate convergence.

Approximate dynamic programming methods such as least-squares policy iteration (LSPI) [12, 13] and fitted Q-iteration (FQI) [2] have been applied successfully to stopping problems, particularly in American option pricing. By leveraging simulated price trajectories, Longstaff and Schwartz [18] introduced the Least Squares Monte Carlo (LSM) algorithm, which estimates the conditional expectation of the continuation value at each time step using cross-sectional ordinary least squares regression over a set of basis functions. This regression-based approach to approximating the continuation value bypassed the curse of dimensionality inherent in traditional grid-based dynamic programming, providing a practical bridge between classical methods and modern learning-based architectures. Li et al. [17] established finite-sample error bounds for LSM, FQI, and LSPI in the context of American option pricing, proving that these methods achieve polynomial sample complexity under standard regularity conditions.

More recently, deep learning has enabled the solution of high-dimensional single stopping problems. Becker et al. [3] introduced deep optimal stopping, using neural networks to approximate continuation values or stopping rules directly from simulated trajectories, and proved convergence under Lipschitz regularity assumptions. Damera Venkata and Bhattacharyya [15] proposed an optimal stopping policy gradient algorithm that leverages recurrent neural networks (RNNs) to handle non-Markovian optimal stopping problems where the decision depends on the full price history rather than just the current state.

While the literature on learning-based methods for single optimal stopping is extensive, work applying reinforcement learning to *multiple* optimal stopping remains limited. Recent efforts have primarily focused on extending deep learning architectures to handle the increased complexity. Lapeyre and Lelong [14] and Huré et al. [9] developed neural network approaches for Bermudan options and high-dimensional nonlinear PDEs arising in multiple exercise problems, using backward schemes. Rouillard et al. [21] proposed a deep Q-learning formulation for optimal stopping with multiple exercises, demonstrating empirical scalability but without theoretical guarantees or systematic comparison to classical methods in low-data regimes. Most recently, Laurière and Talbi [16] introduced a deep learning framework specifically designed for multiple optimal stopping in high dimensions, utilizing backward induction combined with the dynamic programming principle to learn the multidimensional value surface.

However, none of these prior works systematically study tabular or linear function approximation methods for multiple optimal stopping in the offline reinforcement learning setting nor do they empirically benchmark against classical regression-based methods like LSM in data-scarce regimes. A modern comprehensive reference on classical multiple stopping theory is Sofronov and Szajowski [34], though it does not address the reinforcement learning formulation or data-driven policy learning, which are the focus of our work.

The rest of the paper is organized as follows. Section 3 introduces the mathematical preliminaries, formalizing the optimal multiple stopping problem as an augmented-state Markov Decision Process and outlining its fundamental structural properties. Section 4 details the offline tabular and regression-based reinforcement learning frameworks, reviewing baseline algorithms and introducing our proposed Bootstrapped Least-Squares Monte Carlo (TD-LSM) algorithm. Section 5 establishes rigorous finite-sample theoretical bounds for LSPI in multiple stopping setting. Section 6 presents the empirical evaluation of these tabular methods across various simulated environments. Section 7 extends these methodologies to high-dimensional state spaces using deep reinforcement learning architectures. Finally, Section 8 concludes the paper and discusses directions for future work. Detailed proofs, algorithm pseudo codes, and experimental configurations are deferred to the Appendices.

3. Preliminaries

To rigorously analyze and solve optimal stopping problems using reinforcement learning, we first formalize them as discrete-time, finite-horizon Markov Decision Processes (MDPs). We begin by reviewing the classical single stopping problem, and then extend this framework to optimal multiple stopping by explicitly augmenting the state space to track the number of remaining exercise rights.

We consider a finite-horizon MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma, T)$, where $\mathcal{S}$ is the state space, $\mathcal{A} = \{0, 1\}$ with 0 denoting *continue* and 1 denoting *exercise*, $\mathcal{P}(\cdot \mid s, a)$ is the transition kernel, $r(s, a)$ is the immediate reward function,$\gamma \in (0, 1]$ is the discount factor, and $T < \infty$ is the deterministic terminal time horizon. Let $\{Y_t\}_{t=0}^{T}$ denote the state process evolving according to the transition dynamics $\mathcal{P}$, and let $\mathcal{F}_t = \sigma(Y_0, \ldots, Y_t)$ denote its natural filtration.

In a standard MDP framework, a policy $\pi = (\pi_0, \ldots, \pi_{T-1})$ dictates the decision-making strategy. The agent's fundamental objective that we consider is to find a policy that maximizes the expected cumulative discounted reward over the finite horizon [19]:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r(Y_t, a_t) \mid Y_0 = y_0 \right].$$

The optimal policy π^* is the one that attains the supremum over all valid policies, yielding the optimal expected return: $J^*(y_0) = \sup_\pi J(\pi)$. To solve this sequentially, the global objective is broken down into local state-value functions $V_t^\pi(s)$, which represent the expected return from time t onwards under policy π:

$$V_t^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=t}^{T} \gamma^{k-t} r(Y_k, a_k) \mid Y_t = s \right].$$

The corresponding optimal value function is defined as the supremum over all policies starting from state s at time t, $V_t^*(s) = \sup_\pi V_t^\pi(s)$. By the principle of dynamic programming, $V_t^*(s)$ satisfies the general Bellman optimality equation:

$$V_t^*(s) = \max_{a \in \mathcal{A}} \left\{ r(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{P}(\cdot|s,a)}[V_{t+1}^*(s')] \right\}$$

The optimal policy π^* is recovered by acting greedily with respect to $V_t^*(s)$ at each step. Because the problem has a strict finite horizon T, the process is forced to terminate at $t = T$, yielding the terminal boundary condition $V_T^*(s) = \max_a r(s, a)$.

3.1. Optimal Single and Multiple Stopping

In the classical single optimal stopping problem, the agent is granted exactly one exercise opportunity. Taking the exercise action ($a = 1$) yields an immediate payoff and transitions the MDP into a reward-free absorbing state, formally terminating the decision process. Mathematically, the agent's goal is to find a *stopping time* τ that maximizes the expected discounted reward. A stopping time is a random variable $\tau \in \{0, 1, \ldots, T\}$ such that the decision to stop at time t (the event $\{\tau = t\}$) depends exclusively on the information observed up to that point (i.e., $\{\tau = t\} \in \mathcal{F}_t$). This measurability condition ensures that the policy is non-anticipatory, preventing the agent from acting on future information. The objective is to determine an optimal stopping time τ^* that attains the supremum over all valid stopping times:

$$V_0^*(Y_0) = \sup_\tau \mathbb{E} \left[\sum_{t=0}^{\tau-1} \gamma^t r(Y_t, 0) + \gamma^\tau r(Y_\tau, 1) \right]$$

. By evaluating the problem from the terminal horizon T backwards, the global supremum translates into a sequence of local binary decisions. The optimal value function $V_t(s)$ (dropping the asterisk for brevity) satisfies the specific Bellman optimality equation:

$$V_t(s) = \max \left\{ r(s, 1),\ r(s, 0) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, 0)\, V_{t+1}(s') \right\}$$

From this recursive formulation, the optimal stopping time naturally emerges as the first instance when the immediate payoff of exercising meets or exceeds the expected value of continuing (the continuation value):

$$\tau^* = \inf\left\{t : r(Y_t, 1) \geq r(Y_t, 0) + \gamma \mathbb{E}[V_{t+1}(Y_{t+1}) \mid \mathcal{F}_t]\right\}$$

. We now extend this framework to accommodate optimal multiple stopping, where the agent is endowed with $K > 1$ stopping opportunities. Rather than a single stopping time, the agent must define a sequence of ordered stopping times $0 \leq \tau_1 < \tau_2 < \cdots < \tau_K \leq T$. To solve this problem as an MDP, we augment the state space to explicitly track the remaining exercise rights. We define the augmented state as $S_t = (Y_t, K_t) \in \mathcal{S} \times \{0, 1, \ldots, K\}$, where K_t tracks the number of available stops at time t. The dynamics of the remaining rights strictly follow $K_{t+1} = (K_t - A_t)^+$ where $(x)^+ = \max\{x, 0\}$.

To encompass various multiple-stopping scenarios, we denote the reward function by $r(s, k, a)$. Let $h(s)$ represent the intermediate dividend or holding cost when the agent chooses not to exercise ($a = 0$). Let $g_k(s)$ represent the terminal payoff upon exercising ($a = 1$), which depends on both the current state s and the number of remaining rights k. This leads us to the following notion of the reward function for the problem:

$$r(s, k, a) = \begin{cases} g_k(s), & a = 1,\ k > 0, \\ h(s), & a = 0, \\ 0, & k = 0. \end{cases}$$

In many practical settings, the exercise payoff $g_k(s)$ can be simplified as a multiple of a base payoff $g(s)$, yielding $g_k(s) = \alpha_k g(s)$. Here, the scalar α_k encodes the specific exercise rules based on the remaining rights. This allows us to express the reward compactly using indicator functions:

$$r(s, k, a) = \mathbf{1}_{\{a=1, k>0\}} \alpha_k g(s) + \mathbf{1}_{\{a=0\}} h(s).$$

In this paper, we particularly focus on the following two important abstractions of the multi-exercise settings:

- **Repeated-sell:** The agent receives the same payoff upon every exercise. This is recovered by choosing $\alpha_k \equiv 1$, yielding $g_k(s) = g(s)$.
- **Buy–sell:** The agent alternates between taking long and short positions, resulting in alternating cash flow signs. This is achieved by choosing $\alpha_k = (-1)^{k+1}$, yielding $g_k(s) = (-1)^{k+1} g(s)$.

The optimal value function denoted by $V_t(s, k)$ for $0 < t < T$ now satisfies the following Bellman recursion:

$$V_t(s, k) = \max_{a \in \{0,1\}} \left\{ r(s, k, a) + \gamma \sum_{s'} \mathcal{P}(s' \mid s, a)\, V_{t+1}\big(s', (k - a)^+\big) \right\}$$

with $V_t(s, 0) \equiv 0$. Now we introduce the state-action value function (Q-function) and the state action Bellman optimality equations below. Define the state–action value function

$$Q_t(s, k, a) = r(s, k, a) + \gamma \mathbb{E}\big[V_{t+1}(S_{t+1}, (k - a)^+) \mid S_t = s\big]$$

so that $V_t(s, k) = \max_a Q_t(s, k, a)$. Define $C_t(s, k) := \gamma\, \mathbb{E}\big[V_{t+1}(S_{t+1}, k) \mid S_t = s\big]$ as the *continuation value* which is the expected discounted value of *not* exercising at time t with k rights remaining. Then the two Q-values are

$$\begin{aligned} Q_t(s, k, 0) &= r(s, k, 0) + C_t(s, k), \\ Q_t(s, k, 1) &= r(s, k, 1) + \gamma\, \mathbb{E}\big[V_{t+1}(S_{t+1}, k - 1) \mid S_t = s\big] \end{aligned}$$

The use of Q values is particularly amenable to reinforcement learning algorithms.

3.2. Structural Properties

In this section, We establish structural properties of the value function and optimal policy focusing particularly on the *Repeated-Sell* and *Buy-Sell* environments. These properties provide theoretical basis for our learning algorithms as well as the numerical observations. The proofs are deferred to Appendix A.

We first begin with the definitions of monotonicity and modularity required for our analysis:

- **Weakly Increasing** A function $v : \mathcal{S} \to \mathbb{R}$ is weakly increasing if for any states $s_1, s_2 \in \mathcal{S}$ such that $s_1 \geq s_2$, we have $v(s_1) \geq v(s_2)$.
- **Stochastic Monotonicity:** A transition kernel $\mathcal{P}$ is stochastically monotone if for any $s_1 \geq s_2$ and any threshold $x \in \mathcal{S}$:

$$\sum_{s' \geq x} \mathcal{P}(s' \mid s_1) \geq \sum_{s' \geq x} \mathcal{P}(s' \mid s_2)$$

  Equivalently, $\mathcal{P}$ preserves monotonicity: for any weakly increasing function v, the conditional expectation $\mathbb{E}[v(S_{t+1}) \mid S_t = s]$ remains weakly increasing in s.
- **Supermodularity and Submodularity:** For a binary action space $\mathcal{A} = \{a_1, a_2\}$ where $a_2 > a_1$, a function $f : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is supermodular in (s, a) if the difference $f(s, a_2) - f(s, a_1)$ is weakly increasing in s. Conversely, it is submodular if this difference is weakly decreasing in s.

We now state the structural assumptions on the reward and transition dynamics:

(A1) Monotone Rewards: For any $k \in \mathcal{K}$ and $a \in \mathcal{A}$, the reward $r(s, k, a)$ is weakly increasing in s. In our formulations, this requires $g(s)$ and $h(s)$ to be non-decreasing functions.

(A2) Stochastically Monotone Transitions: For every action $a \in \mathcal{A}$, the transition kernel $\mathcal{P}(\cdot \mid s, a)$ is stochastically monotone.

(A3) Supermodular (or Submodular) Q-function: The state-action value function $Q_t(s, k, a)$ must be supermodular in (s, a) for optimal selling, and submodular for optimal buying. Because the underlying state transitions do not depend on the stopping action (i.e., $\mathcal{P}(s' \mid s, 1) = \mathcal{P}(s' \mid s, 0) = \mathcal{P}(s' \mid s)$), this simplifies to requiring the *stopping premium*, defined as:

$$\Delta Q_t(s, k) := Q_t(s, k, 1) - Q_t(s, k, 0)$$

to be weakly increasing in s (supermodular) for a *sell* threshold, or weakly decreasing in s (submodular) for a *buy* threshold.

These assumptions are typical in the stochastic control literature for establishing the optimality of threshold policies [39]. While these properties have been extensively established for optimal single stopping problems, we extend them here in an analogous way to characterize the structure of optimal multiple stopping policies.

Building on the monotonicity and modularity assumptions established above, we first identify the fundamental structural properties of the value function, which are essential for characterizing the optimal policy.

Lemma 1 (Structural Properties of the Value Function)**.** *Under Assumptions A1, A2, and A3, the value function $V_t(s, k)$ satisfies the following structural properties for all times t, states s, and remaining exercise rights $k \geq 1$:*

1. ***Monotonicity in remaining exercise rights:***
   - *Repeated-Sell: Increasing the allowance of exercise rights strictly expands the set of valid policies, meaning the optimal expected reward cannot decrease: $V_t(s, k+1) \geq V_t(s, k)$.*

- *Buy-Sell: Because buy-sell must alternate, monotonicity holds amongst pairs of buy rights and pairs of sell rights:* $V_t(s, k+2) \geq V_t(s, k)$.

2. ***Time monotonicity:*** *An earlier starting time expands the available stopping horizon, meaning the optimal expected reward cannot decrease for either formulation:* $V_t(s, k) \geq V_{t+1}(s, k)$.

3. ***Diminishing marginal value:***

   - *Repeated-Sell: The marginal increase in expected reward from adding one more exercise right is non-increasing. Defining the sequential marginal value as* $\Delta_t(s, k) := V_t(s, k) - V_t(s, k-1)$*, it holds for* $k \geq 2$ *that:* $\Delta_t(s, k) \leq \Delta_t(s, k-1)$
   - *Buy-Sell: Because individual rights alternate between buying and selling, concavity holds over complete buy-sell cycles. Defining the marginal cycle value as* $\Delta_t^{cycle}(s, k) := V_t(s, k) - V_t(s, k-2)$*, it holds for* $k \geq 3$ *that the benefit of an additional buy-sell trade satisfies:* $\Delta_t^{cycle}(s, k) \leq \Delta_t^{cycle}(s, k-2)$

These structural properties directly dictate the structure of the optimal policy. Specifically, the supermodularity/submodularity of the stopping premium ensures that the optimal decision boundaries are defined by state-dependent thresholds. Furthermore, the diminishing marginal value established in Lemma 1 mathematically guarantees that the opportunity cost of exercising decreases when the agent holds an abundance of rights. By generalizing the standard single-stop formulation to an arbitrary number of rights K, we can formally characterize the sequential threshold policies for both repeated-sell and buy-sell environments.

Lemma 2 (Threshold-Type Optimal Policies)**.** *Under Assumptions A1, A2, and A3, the optimal stopping rule is characterized by time-dependent thresholds* θ_t^k*, where* $k \in \{1, \ldots, K\}$ *denotes the number of remaining rights. Let* τ_k *denote the sequential stopping time at which exactly k rights remain (such that* $\tau_K < \tau_{K-1} < \cdots < \tau_1$*). The optimal policies are characterized as follows:*

1. ***Repeated-Sell:*** *The sequential stopping times are given by:*

$$\tau_k = \inf\left\{t > \tau_{k+1} : g(Y_t) - h(Y_t) \geq \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) - V_{t+1}(S_{t+1}, k-1) \mid S_t = Y_t]\right\},$$

   where $\tau_{K+1} \equiv -1$*. Due to the diminishing marginal value established in Lemma 1, the opportunity cost of exercising is lower when holding more rights. This results in a nested policy structure where the threshold for earlier exercises creates a larger exercise region than for later ones (*$\theta_t^K \leq \theta_t^{K-1} \leq \cdots \leq \theta_t^1$*).*

2. ***Buy-Sell:*** *Assuming K is even (initiating with a buy position), the sequential stopping times alternate between buying (even k) and selling (odd k):*

$$\tau_k(Buy) = \inf\left\{t > \tau_{k+1} : g(Y_t) \leq \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-1) - V_{t+1}(S_{t+1}, k) \mid S_t = Y_t]\right\},$$
$$\tau_{k-1}(Sell) = \inf\left\{t > \tau_k : g(Y_t) \geq \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-1) - V_{t+1}(S_{t+1}, k-2) \mid S_t = Y_t]\right\}.$$

   The structural constraint of alternating actions introduces a hysteresis band ($\theta_t^{buy} \leq \theta_t^{sell}$*), defining the minimum price spread required to justify entering and subsequently exiting a position.*

4. Tabular Reinforcement Learning in Optimal Multiple Stopping

In this section, we formulate the optimal multiple stopping problem within a reinforcement learning framework. In our offline setting, the underlying asset price process $\{S_t\}_{t=0}^T$ evolves independently of whether the agent exercises or holds. We therefore do not rely on an explicit behavioral policy to generate action-conditioned transitions. Instead, for both the local regression-based methods (LSM and TD-LSM) and the global function approximation methods (FQI and LSPI), we train directly on the raw simulated price paths $\{s_{n,t}\}_{n=1}^{N_{\text{train}}}$. Because the immediate payoff of exercising is known deterministically, these algorithms implicitly evaluate the hold and exercise decisions at every node along the paths without requiring an

exhaustive enumeration of (s, a, r, s') tuples. This allows the algorithms to safely extract the maximum possible signal directly from the fixed paths without any online interaction with the market.

In the rest of this section, we present a set of algorithms designed to evaluate and optimize multiple stopping policies, starting from tabular and linear approximations, and extending to deep reinforcement learning architectures in later sections.

4.1. Baseline Algorithms

We begin with foundational algorithms that handle finite state spaces or utilize linear basis-function approximations to estimate value functions.

4.1.1. The Longstaff–Schwartz Method (LSM)

The Longstaff–Schwartz Method [18] was originally introduced as a regression-based approximate dynamic programming scheme for pricing American-style options. While originally pioneered for the valuation of American-style options in financial engineering, the core regression-based framework of the Longstaff–Schwartz Method is highly versatile and can be readily adapted to solve any general optimal stopping problem formulated as a Markov Decision Process In the reinforcement learning context, LSM operates on a fixed offline dataset of Monte Carlo trajectories to estimate the optimal stopping policy backward in time.

For a single exercise right, the state–action value functions are:

$$Q_t(s, 1) = r(s, 1)$$

$$Q_t(s, 0) = r(s, 0) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}) \mid S_t = s]$$

While the immediate exercise value $Q_t(s, 1)$ is known deterministically, the continuation value $Q_t(s, 0)$ involves an intractable conditional expectation. LSM approximates this continuation value using cross-sectional ordinary least squares (OLS) regression. We define a feature map $\phi : \mathcal{S} \to \mathbb{R}^d$ that transforms the state (e.g., the M-dimensional asset price vector) into a d-dimensional basis representation (such as Laguerre polynomials). The continuation Q-value is approximated as a linear combination of these basis functions:

$$\widehat{Q}_t(s, 0) = \sum_{j=1}^{d} \beta_{t,j}\, \phi_j(s) = \phi(s)^\top \beta_t$$

The parameter vector $\beta_t \in \mathbb{R}^d$ is found by regressing the realized discounted future returns, $\gamma V_{n,t+1}$, onto the features $\phi(s_{n,t})$. Crucially, LSM restricts this regression strictly to "In-The-Money" (ITM) paths: trajectories where the immediate exercise payoff is strictly positive ($r(s_{n,t}, 1) > 0$). This localization ensures that the regression capacity is dedicated entirely to accurately fitting the boundary where the agent must actively choose between holding and exercising, completely ignoring out-of-the-money states where exercising is trivially suboptimal.

4.1.2. Extension to Optimal Multiple Stopping

To adapt LSM for the multiple stopping framework, we must account for the augmented state (S_t, K_t). Recall that with $k \geq 1$ remaining rights, the Q-values become:

$$Q_t(s, k, 1) = r(s, k, 1) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k - 1) \mid S_t = s]$$

$$Q_t(s, k, 0) = r(s, k, 0) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s]$$

Unlike the single-stop setting where exercising yields a deterministic terminal payoff, the multiple-stop exercise action $Q_t(s, k, 1)$ now includes a residual continuation value for the $k - 1$ remaining rights. To solve this systematically, our extended LSM (outlined in Algorithm 1) introduces an outer loop iterating forward over the rights levels $k = 1, \ldots, K$, and an inner loop iterating backward over time $t = T - 1, \ldots, 0$.

At each time step t and rights level k, we maintain two separate linear regressions:

- **Continuation regression:** Estimates the value of holding, $\widehat{Q}_t(s,k,0) = \phi(s)^\top \beta^{\text{cont}}$, using $\gamma V_{n,t+1}(k)$ as the target.
- **Residual regression:** Estimates the expected additional value of retaining $k-1$ rights after an immediate exercise, $\widehat{Q}_t^{\text{ex-residual}}(s) = \phi(s)^\top \beta^{\text{ex}}$, using $\gamma V_{n,t+1}(k-1)$ as the target.

Because the outer loop processes rights in ascending order, the values $V_{n,t+1}(k-1)$ for the residual regression are already fully computed from the previous $(k-1)$-th iteration and act as stationary targets. The agent then decides to exercise if the total estimated exercise value exceeds the estimated continuation value: $r(s_{n,t},k,1) + \widehat{Q}_t^{\text{ex-residual}}(s_{n,t}) > \widehat{Q}_t(s_{n,t},k,0)$.

Algorithm 1 Longstaff–Schwartz Method (LSM) for Optimal Multiple Stopping

Hyperparameters: Basis functions $\Phi(s) \in \mathbb{R}^d$, Discount factor γ, Min samples $N_{\min}$
Input: Set of N paths $\{s_{n,t}\}_{n=1}^N$ for $t = 0, \dots, T$.
Initialize: Policy dictionary $\mathcal{W} = \emptyset$ to store regression weights.
for $k = 1$ **to** K **do**
  Initialize realized values $V_n \leftarrow \text{Payoff}(s_{n,T}, k)$ for all paths $n \in \{1, \dots, N\}$.
  for $t = T-1$ **down to** 1 **do**
    Let $\mathcal{I}_{\text{ITM}} = \{n \in [N] \mid \text{Payoff}(s_{n,t}, k) > 0\}$.
    if $|\mathcal{I}_{\text{ITM}}| < N_{\min}$ **then**
      $V_n \leftarrow \gamma V_n$ for all n ▷ Skip regression if ill-conditioned
      continue
    end if
    Target Construction:
    for $n \in \mathcal{I}_{\text{ITM}}$ **do**
      $y_n^{\text{cont}} \leftarrow \gamma V_n$ ▷ Realized continuation
      if $k > 1$ **then**
        res $\leftarrow$ QueryModel$(\mathcal{W}, s_{n,t+1}, k-1)$ ▷ Evaluate $(k-1)$ value at $t+1$
        $y_n^{\text{ex}} \leftarrow \gamma \times$ res
      else
        $y_n^{\text{ex}} \leftarrow 0$
      end if
    end for
    Regression Step:
    Regress y^{cont} onto $\Phi(s_{n,t})$ for $n \in \mathcal{I}_{\text{ITM}}$ to obtain weights $\beta_{t,k}^{\text{cont}}$.
    Regress y^{ex} onto $\Phi(s_{n,t})$ for $n \in \mathcal{I}_{\text{ITM}}$ to obtain weights $\beta_{t,k}^{\text{ex}}$.
    Store $\mathcal{W}[(k,t,\text{'cont'})] \leftarrow \beta_{t,k}^{\text{cont}}$ and $\mathcal{W}[(k,t,\text{'ex'})] \leftarrow \beta_{t,k}^{\text{ex}}$.
    Decision & Update Step:
    for $n \in \mathcal{I}_{\text{ITM}}$ **do**
      $\widehat{Q}^{\text{hold}} \leftarrow \Phi(s_{n,t})^\top \beta_{t,k}^{\text{cont}}$
      $\widehat{Q}^{\text{ex}} \leftarrow \text{Payoff}(s_{n,t}, k) + \Phi(s_{n,t})^\top \beta_{t,k}^{\text{ex}}$
      if $\widehat{Q}^{\text{ex}} > \widehat{Q}^{\text{hold}}$ **then**
        $V_n \leftarrow \text{Payoff}(s_{n,t}, k) + y_n^{\text{ex}}$ ▷ Update with realized exercise
      else
        $V_n \leftarrow y_n^{\text{cont}}$ ▷ Update with realized continuation
      end if
    end for
    For all $n \notin \mathcal{I}_{\text{ITM}}$, update $V_n \leftarrow \gamma V_n$.
  end for
end for
Return $\mathcal{W}$

4.1.3. Justification for the Dual-Regression Architecture

A critical distinction between single and multiple optimal stopping lies in how the exercise action is evaluated. In a classical single-stop setting, the exercise decision terminates the MDP and yields a known, deterministic immediate payoff, requiring only the continuation branch to be estimated via regression. However, in the multiple-stopping framework, exercising with $k > 1$ rights yields an immediate payoff plus the residual option value of holding $k-1$ rights. A naive approach might simply evaluate the already-fitted $(k-1)$ value function at the realized next-step state, S_{t+1}, and add it to the immediate payoff. However, doing so constitutes "peeking" into the future: it uses S_{t+1} to make a decision at time t, fundamentally violating the strict non-anticipatory ($\mathcal{F}_t$-measurable) requirement of stopping times. To ensure the policy remains strictly causal and relies only on information available at time t, we must compute the true conditional expectation of this future residual value. This necessitates a second, independent regression for the exercise branch.

While this theoretical formulation is expressed in terms of Q-values to align with standard RL literature, Algorithm 1 practically implements this by tracking and updating the realized value function $V_{n,t}(k)$ for each trajectory n.

Remark 1. While Algorithm 1 is presented assuming the agent can exercise rights at consecutive time steps without restriction, it is not hard to extend this framework to accommodate a mandatory refractory period $\delta > 0$. When a lockout constraint exists, an exercise action at time t defers the next available decision epoch to $t_{\text{resume}} = \min(t + \delta + 1, T)$. Algorithmically, one simply replaces the evaluation of the $k-1$ residual target at $t+1$ with an evaluation at t_{resume} and adjusts the discount factor to $\gamma^{t_{\text{resume}}-t}$ accordingly.

4.1.4. Fitted Q-Iteration (FQI)

Fitted Q-Iteration [2] approximates the optimal Q-function by iteratively applying the Bellman optimality operator to a fixed dataset. Because the underlying state transitions (asset prices) in our optimal stopping environment are exogenous and independent of the agent's actions, we do not require a traditional offline RL dataset of explicit (s, a, r, s') tuples collected by a behavioral policy. Instead, our dataset simply comprises the N_{train} uncontrolled price trajectories. The algorithm implicitly evaluates both the "hold" and "exercise" actions at every state transition (S_t, S_{t+1}) analytically.

Following the exact linear architecture proposed by Li et al. [17] for American options, we parameterize the continuation value as $\widehat{Q}(s, t, 0) = \phi(s, t)^\top w$, where $\phi(s, t) \in \mathbb{R}^7$ is a fixed 7-dimensional feature vector. This vector concatenates three Laguerre spatial features of the normalized price s/K, three temporal features (sinusoidal, logarithmic, and quadratic functions of t/N), and a constant term.

Unlike LSM, which isolates the state space at each specific time step t and solves T separate regressions backward in time, FQI is "global" in the sense that it trains a single weight vector w that parameterizes the Q-function across the entire state-time space simultaneously.

To adapt FQI for optimal multiple stopping, we sequentially train a separate global weight vector w_k for each rights level $k \in \{1, \ldots, K\}$ in ascending order. For a fixed level k, at iteration n, a regression target $y_{i,t}^{(n)}$ is computed for every transition $(S_{i,t}, S_{i,t+1})$ in the dataset using the current weight estimate $w_k^{(n-1)}$ and the previously fixed lower-level weights w_{k-1}:

$$y_{i,t}^{(n)} = \gamma \max \left\{ r(S_{i,t+1}, k, 1) + \phi(S_{i,t+1}, t+1)^\top w_{k-1},\ \phi(S_{i,t+1}, t+1)^\top w_k^{(n-1)} \right\}$$

where $w_0 \equiv \mathbf{0}$. This target represents the discounted maximum of exercising at $t+1$ (yielding the immediate payoff plus the estimated residual value of $k-1$ rights) versus continuing at $t+1$ with k rights.

The weight vector is then updated by solving an ordinary least squares problem across all paths and all time steps $t \in \{1, \ldots, T-1\}$:

$$w_k^{(n)} = \arg\min_{w \in \mathbb{R}^7} \sum_{i,t} \left(y_{i,t}^{(n)} - \phi(S_{i,t}, t)^\top w \right)^2$$

This reduces to the closed-form solution $w_k^{(n)} = (\Phi^\top \Phi)^{-1} \Phi^\top Y^{(n)}$, where Φ is the stacked design matrix of all state-time features. Because $\Phi^\top \Phi$ depends only on the exogenous state features and remains strictly

constant across all iterations, its inverse can be precomputed exactly once, making the iterative updates highly computationally efficient [17].

4.1.5. Least-Squares Policy Iteration (LSPI)

Least-Squares Policy Iteration [13] is a linear Approximate Policy Iteration (API) method. It does not utilize neural networks or separate actor-critic architectures; rather, it alternates between evaluating a fixed policy using Least-Squares Temporal Difference (LSTD) and greedily updating that policy.

We again strictly follow the formulation of Li et al. [17], using the same 7-dimensional spatio-temporal feature vector $\phi(s,t)$ introduced in FQI to parameterize the continuation value. As with FQI, we extend LSPI to multiple stopping by training a weight vector w_k sequentially for each rights level k.

For a fixed level k and a fixed policy π, the LSTD step estimates the continuation value by solving the linear system $Aw_k = b$. The matrix $A \in \mathbb{R}^{7\times 7}$ and vector $b \in \mathbb{R}^7$ accumulate the temporal difference statistics across all transitions $(S_{i,t}, S_{i,t+1})$ in the dataset:

$$A = \sum_{i,t} \phi(S_{i,t},t)\Big(\phi(S_{i,t},t) - \gamma \mathbf{1}_{\{\pi(S_{i,t+1})=0\}}\phi(S_{i,t+1},t+1)\Big)^\top$$

$$b = \sum_{i,t} \gamma\phi(S_{i,t},t)\mathbf{1}_{\{\pi(S_{i,t+1})=1\}}\Big(r(S_{i,t+1},k,1) + \phi(S_{i,t+1},t+1)^\top w_{k-1}\Big)$$

Here, the indicator function $\mathbf{1}_{\{\pi=0\}}$ masks out the future feature vector if the policy dictates exercising at $t+1$, correctly reflecting that the continuation process terminates. Conversely, $\mathbf{1}_{\{\pi=1\}}$ accumulates the discounted exercise payoff and the residual continuation value from the lower rights layer w_{k-1}.

After solving $w_k = A^{-1}b$ (often regularized with a small ridge penalty λI to prevent early-iteration singularities), the policy π is updated greedily. The new policy chooses to exercise if the value of exercising strictly exceeds the newly estimated continuation value:

$$\pi_{\text{new}}(s,t) = \mathbf{1}_{\{r(s,k,1)+\phi(s,t)^\top w_{k-1} \,>\, \phi(s,t)^\top w_k\}}$$

This process of rebuilding A and b and updating w_k is iterated until the weight vector converges.

4.2. Proposed method: Bootstrapped Least-Squares Monte Carlo (TD-LSM)

Bootstrapped LSMC (TD-LSM) is a temporal-difference variant of the Longstaff–Schwartz method. As described in Section 4.1.1, standard LSM regresses against the *realized* discounted payoff—the actual accumulated reward observed along the remainder of the specific Monte Carlo trajectory.

TD-LSM, instead, employs a one-step lookahead mechanism. For the regression at time t, the dependent variables (commonly referred to as the regression targets) are constructed using the fitted value estimates strictly from the *next* time step, $t+1$. We define the temporal-difference dependent variable for the continuation regression as:

$$Y_{n,t}^{\text{cont}} = \gamma \widehat{V}_{t+1}(s_{n,t+1}, k)$$

By replacing high-variance Monte Carlo realized returns with these smooth, lower-variance bootstrapped estimates, TD-LSM achieves significantly more stable regression fits. This is especially crucial when the number of available sample paths is small or when environment volatility causes realized trajectory returns to fluctuate wildly.

Algorithm 2 outlines the full procedure. The critical algorithmic departure from standard LSM occurs in the value function update step (Lines 18–25). In standard LSM, the value sequence V_n is updated using the realized path outcome based on the regression's binary decision. In TD-LSM, for in-the-money paths, $\widehat{V}_n$ is updated to be the maximum of the newly fitted *estimated* Q-values. Out-of-the-money paths simply inherit the discounted estimate from the next step.

Algorithm 2 Bootstrapped Least-Squares Monte Carlo (TD-LSM) for Optimal Multiple Stopping

Hyperparameters: Basis functions $\Phi(s) \in \mathbb{R}^d$, Discount factor γ, Min samples $N_{\min}$
Input: Set of N paths $\{s_{n,t}\}_{n=1}^{N}$ for $t = 0, \ldots, T$.
Initialize: Policy dictionary $\mathcal{W} = \emptyset$.
for $k = 1$ **to** K **do**
    Initialize estimated values $\widehat{V}_n \leftarrow r(s_{n,T}, k)$ for all $n \in \{1, \ldots, N\}$.
    for $t = T - 1$ **down to** 1 **do**
        Let $\mathcal{I}_{\text{ITM}} = \{n \in [N] \mid r(s_{n,t}, k) > 0\}$.
        Dependent Variable Construction:
        for $n \in \{1, \ldots, N\}$ **do**
            $Y_n^{\text{cont}} \leftarrow \gamma \widehat{V}_n$ ▷ Bootstrapped continuation target
            if $k > 1$ **then**
                res $\leftarrow$ QueryModel$(\mathcal{W}, s_{n,t+1}, k-1)$ ▷ Evaluate $(k-1)$ value at $t+1$
                $Y_n^{\text{ex}} \leftarrow \gamma \times$ res
            else
                $Y_n^{\text{ex}} \leftarrow 0$
            end if
        end for
        if $|\mathcal{I}_{\text{ITM}}| < N_{\min}$ **then**
            $\widehat{V}_n \leftarrow \max(r(s_{n,t}, k) + Y_n^{\text{ex}},\ Y_n^{\text{cont}})$ for all n
            continue
        end if
        Regression Step:
        Regress Y^{cont} onto $\Phi(s_{n,t})$ for $n \in \mathcal{I}_{\text{ITM}}$ to obtain weights $\beta_{t,k}^{\text{cont}}$.
        Regress Y^{ex} onto $\Phi(s_{n,t})$ for $n \in \mathcal{I}_{\text{ITM}}$ to obtain weights $\beta_{t,k}^{\text{ex}}$.
        Store $\mathcal{W}[(k, t, \text{'cont'})] \leftarrow \beta_{t,k}^{\text{cont}}$ and $\mathcal{W}[(k, t, \text{'ex'})] \leftarrow \beta_{t,k}^{\text{ex}}$.
        Value Function Update Step:
        for $n \in \mathcal{I}_{\text{ITM}}$ **do**
            $\widehat{Q}^{\text{hold}} \leftarrow \Phi(s_{n,t})^\top \beta_{t,k}^{\text{cont}}$
            $\widehat{Q}^{\text{ex}} \leftarrow r(s_{n,t}, k) + \Phi(s_{n,t})^\top \beta_{t,k}^{\text{ex}}$
            $\widehat{V}_n \leftarrow \max(\widehat{Q}^{\text{hold}},\ \widehat{Q}^{\text{ex}})$ ▷ Update using fitted estimates
        end for
        For all $n \notin \mathcal{I}_{\text{ITM}}$, update $\widehat{V}_n \leftarrow Y_n^{\text{cont}}$. ▷ OTM paths inherit continuation value directly
    end for
end for
Return $\mathcal{W}$

5. Theoretical Results

In this section, we develop finite-time bounds on the performance of the policy obtained with LSPI for the optimal multiple stopping problem. The central task is to identify where and how the additional structure of multiple exercise rights in the Bellman operator modifies each term of the bound. Our main finding is that the LSPI algorithm inherits an additional factor of K (the number of exercise rights) in its suboptimality bound, arising from the K linked value layers in the MDP. Our analysis for LSPI closely follows the approach of Li et al. [17], who established analogous results for the *single* stopping problem.

To establish our theoretical framework, we assume a dataset comprising n independently sampled price trajectories, each of length T. This yields a total of $n_0 = nT$ transitions. We define ν_t as the marginal distribution of the price process at time t, representing the cross-sectional distribution of the underlying stock prices across all possible paths at that specific time step. We then write $\nu = (\nu_0, \nu_1, \ldots, \nu_{T-1})$ for the

sequence of marginals, which characterizes the full unconditional evolution of the asset prices over the entire time horizon.

Before detailing the algorithm, we outline the assumption required for analysis:

(A4) Bounded rewards: $0 \le r(s,k,a) \le R_{\max}$ for all (s,k,a). Consequently, $0 \le V_t(s,k) \le Q_{\max} := KR_{\max}/(1-\gamma)$ for all (t,s,k).

By Assumption A4, the maximum achievable discounted total reward with K exercise rights is bounded by $Q_{\max}$. As noted previously, the exercise Q-value $Q_t(s,k,1)$ can be written as the known immediate payoff plus the continuation value *at the lower rights level* $k-1$:

$$Q_t(s,k,1) \;=\; r(s,k,1) \;+\; \gamma\,\mathbb{E}\big[V_{t+1}(S_{t+1},k-1) \mid S_t = s\big]. \tag{1}$$

Because $r(s,k,1)$ is the known payoff function, the entire difficulty of the learning problem reduces to estimating the family of *continuation values* $C_t(s,k) := \gamma\,\mathbb{E}[V_{t+1}(S_{t+1},k) \mid S_t = s]$. In LSPI, this observation allows us to keep one weight vector w and still handle all rights levels simultaneously, exactly as in the single-stopping case where the exercise value is known and only the continuation value is learned.

Li et al. [17] define the function space for single stopping as

$$\mathcal{F}_1 \;=\; \big\{h : \mathcal{S}\times\mathcal{A} \to \mathbb{R} \;\big|\; h(s,1) = r(s,1),\; h(s,0) = \theta^\top\phi(s),\; \theta\in\mathbb{R}^d,\; h(e)=0\big\},$$

where $\phi : \mathcal{S} \to \mathbb{R}^d$ is the feature map and e is the terminal absorbing state ensuring that the value drops to zero upon completion. For the multiple stopping problem under LSPI, the state includes the rights count k, and the Q-function must be represented over the full augmented space. We define the expanded function class as:

$$\begin{aligned}\mathcal{F}_K \;=\; \big\{h : \mathcal{S}'\times\mathcal{A}\to\mathbb{R} \;\big|\; & h(s,k,1) = r(s,k,1) + \gamma w^\top\phi(s,k-1),\\ & h(s,k,0) = w^\top\phi(s,k),\; w\in\mathbb{R}^d,\; h(s,0,a)=0\big\}\end{aligned} \tag{2}$$

where $\phi : \mathcal{S}\times\mathcal{K} \to \mathbb{R}^d$ now incorporates the rights level, and the same weight vector w is shared across all time steps and rights levels. The constraint $h(s,k,1) = r(s,k,1) + \gamma w^\top\phi(s,k-1)$ encodes the separation of immediate payoff and estimated continuation value at level $k-1$, which is itself in $\mathcal{F}_K$. The constraint $h(s,0,a) = 0$ explicitly enforces the boundary condition $V_t(s,0) = 0$ for all t, s, effectively acting as our terminal state condition when rights are exhausted.

The performance of LSPI ultimately depends on how well the function class $\mathcal{F}_K$ can represent the Bellman image of its own elements. Let $T_K^{\pi'}$ denote the Bellman operator for policy π' in the multiple stopping MDP:

$$(T_K^{\pi'}Q)(s,k,a) \;=\; r(s,k,a) + \gamma\,\mathbb{E}\big[Q(S_{t+1},(k-a)^+,\pi'(S_{t+1},(k-a)^+)) \mid S_t = s\big].$$

For any bounded Q-function $Q\in\mathcal{F}_K$, let π'_Q denote the greedy policy that selects the action maximizing Q at each state. To measure approximation quality, we introduce the $L^2(\nu)$ norm, which averages the squared function values over all state-action pairs weighted by the marginal distributions ν_t at each time step. Specifically, for any function $f : \mathcal{S}'\times\mathcal{A}\to\mathbb{R}$ defined on the augmented state-action space, we define

$$\|f\|_\nu^2 \;=\; \frac{1}{T}\sum_{t=0}^{T-1}\mathbb{E}_{S_t\sim\nu_t}\left[\sum_{k\in\mathcal{K}}\sum_{a\in\mathcal{A}} f(S_t,k,a)^2\right].$$

This norm captures the expected squared deviation of f across all combinations of time steps, stock price states (distributed according to ν_t), rights levels, and actions. Since Q-functions and their Bellman images are deterministic functions of the state-action triple (s,k,a), the norm $\|Q - T_K^{\pi'}Q'\|_\nu$ measures the expected squared difference between two Q-functions when states are sampled from the marginal distributions encountered along price trajectories.

We now define the *inherent Bellman error* and *inherent one-step Bellman error* of $\mathcal{F}_K$:

$$\mathcal{E}_\infty(\mathcal{F}_K) \;=\; \sup_{Q'\in\mathcal{F}_K}\ \inf_{Q\in\mathcal{F}_K} \left\|Q - T_K^{\pi'_{Q'}} Q'\right\|_\nu, \tag{3}$$

$$\mathcal{E}_1(\mathcal{F}_K) \;=\; \sup_{Q',Q''\in\mathcal{F}_K}\ \inf_{Q\in\mathcal{F}_K} \left\|Q - T_K^{\pi'_{Q'}} Q''\right\|_\nu \tag{4}$$

The inherent Bellman error $\mathcal{E}_\infty(\mathcal{F}_K)$ quantifies the worst-case approximation error when applying the Bellman operator to a function in $\mathcal{F}_K$ and projecting the result back into $\mathcal{F}_K$. It measures the fundamental representational limitation of the function class: even if we could compute the exact Bellman backup $T_K^{\pi'_{Q'}} Q'$ for some $Q' \in \mathcal{F}_K$, the result may not lie in $\mathcal{F}_K$, forcing us to approximate it with some $Q \in \mathcal{F}_K$. The inherent one-step Bellman error $\mathcal{E}_1(\mathcal{F}_K)$ extends this concept by allowing the Bellman operator to be applied to a potentially different function $Q'' \in \mathcal{F}_K$ than the one defining the greedy policy $\pi'_{Q'}$, capturing cross-contamination between policy evaluation and improvement steps. We then combine these to define the total approximation error:

$$\mathcal{E}(\mathcal{F}_K) \;=\; \Big(\mathcal{E}_\infty^2(\mathcal{F}_K) + \mathcal{E}_1^2(\mathcal{F}_K)\Big)^{1/2}. \tag{5}$$

When $K = 1$ and all states with zero rights are merged into a single exit state, the operator $T_K^{\pi'}$ reduces to the standard single-stopping Bellman operator, meaning our multiple-stopping quantities act as a strict generalization.

Let $\mathcal{S}_c = \{(s, k, 0) : s \in \mathcal{S},\ k \in \mathcal{K}\} \subset \mathcal{S}' \times \mathcal{A}$ denote the set of "hold" state-action pairs, representing all states where the agent chooses not to exercise. The bound below characterizes the difference in performance between the optimal Q-function and the policy learned after M iterations, restricted to these continuation states.

Theorem 3 (Bound on the performance of LSPI for multiple stopping)**.** *Execute LSPI with a training set of n price trajectories of length T ($n_0 = nT$ total transitions), feature dimension d, and $M > 0$ policy iterations. Let $\pi_{M,n}$ be the greedy policy with respect to the Q-function obtained after M iterations. Then, for any $0 < \delta < 1$, with probability at least $1 - \delta$,*

$$\left\|(Q^* - Q^{\pi_{M,n}})\big|_{\mathcal{S}_c}\right\|_\nu \;\le\; \frac{2\gamma}{(1-\gamma)^2}\Big(K\cdot\mathcal{E}(\mathcal{F}_K) \;+\; \Delta_n \;+\; \gamma^{M/2} Q_{\max}\Big). \tag{6}$$

Here

$$\Delta_n \;=\; C\left(\frac{\Lambda\max(\Lambda,1)}{n}\right)^{1/4}, \qquad \Lambda \;=\; d\log\frac{n}{1-\gamma} + \log\frac{M}{\delta} + T, \tag{7}$$

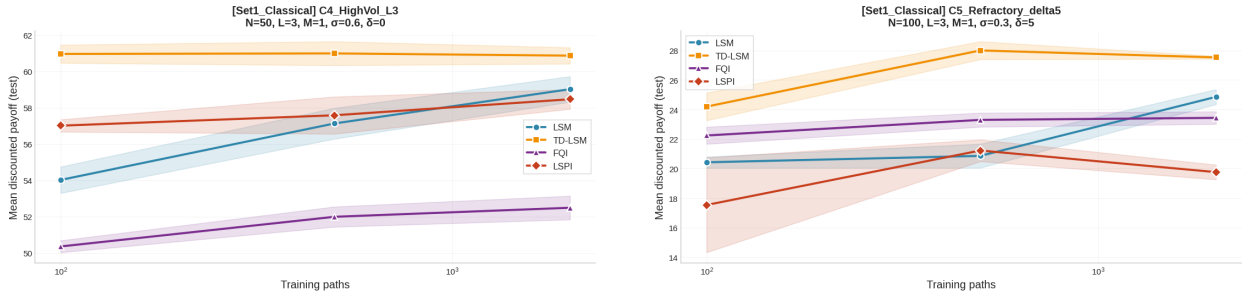
where $C > 0$ is a universal constant.

The proof of Theorem 3 is analogous to Li et al. [17], but it differs from it in exactly two places: (i) the approximation error term acquires the factor K, becoming $K \cdot \mathcal{E}(\mathcal{F}_K)$, and (ii) $R_{\max}$ is replaced by $Q_{\max} = KR_{\max}/(1-\gamma)$. The statistical term Δ_n retains the same form and the same rate $n^{-1/4}$; only Λ changes to include the rights-level in the logarithmic factor. The bound shows that with sufficient data (n large), enough policy iterations (M large), and a well-chosen function class (small $\mathcal{E}(\mathcal{F}_K)$), the learned policy $\pi_{M,n}$ approaches the optimal policy performance, with the suboptimality gap scaling linearly with the number of exercise rights K.

Convergence analysis for the LSM and TD-LSM algorithms, which operate under different structural assumptions and backward induction schemes, will be considered in future work.

6. Experimental Results: Tabular Setting

To benchmark the performance of the proposed methods against standard approaches, we evaluate the algorithms within a custom offline option pricing environment. Detailed formulations of the environment



(a) **High-Volatility Swing** (N=50, L=3, σ=0.6, δ=0). TD-LSM (orange) maintains a payoff of ≈ 61 across all training sizes, while LSM (blue) starts at ≈ 54 and FQI (brown) stagnates around 52.

(b) **Refractory Swing** (N=100, L=3, σ=0.3, δ=5). TD-LSM leads by $\approx$ 4–8 points.

Figure 1: Expected discounted return (test set) vs. number of training paths for the American Put configurations. Shaded bands denote ± 1 standard deviation over five seeds. The x-axis is log-scaled.

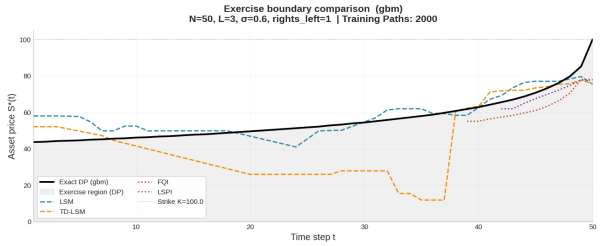
dynamics, state space representations, and hyperparameters are provided in Appendix C.1 and Appendix C.2. We evaluate four tabular algorithms in the offline optimal multiple stopping setting: LSM, FQI, LSPI, and our proposed TD-LSM. All methods use degree-3 Laguerre polynomial basis functions and are trained on a fixed offline dataset of GBM paths. Performance is measured as the expected discounted return on a held-out test set of 10,000 paths, averaged over five independent runs. The training set size varies from 10^2 to 3×10^4 paths to probe sample efficiency.

We consider multiple problem configurations spanning standard American puts, multi-stop swing options, high-volatility regimes, refractory periods, and basket options with their details given in Appendix C. All options use a basket put payoff function, paying $\max(K - \bar{S}_t, 0)$ upon exercise, where $\bar{S}_t$ is the mean price across the M underlying asset processes. Here we focus on the two configurations that differentiate the algorithms. Figure 1 reports the expected discounted returns for the following configurations: a high-volatility swing option (a standard multi-exercise option in a turbulent market with high variance) and a swing option with a refractory period (which imposes a mandatory δ-step waiting period between consecutive exercises). Figure 2 shows the learned exercise boundaries, the critical price thresholds $S^*(t)$ below which it is optimal to exercise, for the former case. Across both settings TD-LSM achieves the highest payoff at every training size, with the largest advantage in the small-data regime.

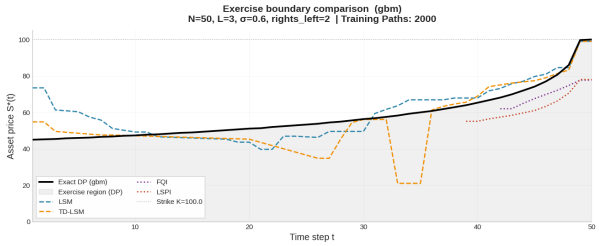
At high volatility, GBM paths move rapidly across the strike price, creating two structural difficulties for the baseline algorithms. First, the fraction of ITM paths at any given time step fluctuates a lot, so the ITM subsample used in LSM's regression may be very small or not a good representation of the full state distribution. Second, the realized continuation values, used as regression targets in LSM, carry enormous variance: a single path's outcome from step t to expiry depends on many large, noisy increments, and this variance propagates directly into the learned boundary.

TD-LSM addresses both issues through its *TD-bootstrapped target*. Rather than regressing on realized payoffs, at each step t it forms the target $\hat{y}_t = \gamma \hat{V}_{t+1}$, where $\hat{V}_{t+1}$ is the fitted regressor at step $t+1$, a smooth polynomial surface that has already been regularised by least squares. Because $\hat{V}_{t+1}$ is smooth, the target is substantially less variable than the corresponding realized path outcome, regardless of how noisy the underlying paths are. Moreover, TD-LSM applies its value update across *all* paths, not only ITM paths, so the out-of-the-money (OTM) region (where immediate exercise yields zero payoff, $\bar{S}_t \geq K$) of the state space is also covered. This is critical at $\sigma = 0.6$ where many paths spend significant time out of the money. The boundary plots (Figure 2) show that LSM's boundary (blue) oscillates wildly and collapses deep into the exercise region mid-trajectory, whereas TD-LSM (orange) remains smooth and close to the exact DP boundary (black) at all three rights levels. As a result, TD-LSM achieves a mean payoff of $\approx$61 across all training sizes, compared to $\approx$54–59 for LSM, $\approx$50–52 for FQI, and $\approx$57–58.5 for LSPI (Figure 1a).

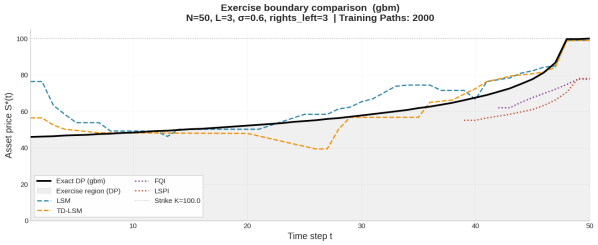
The refractory constraint imposes a mandatory δ-step waiting period after each exercise: upon exercising right ℓ at time t, the agent cannot exercise again until step $t+\delta+1$. This creates a long-range dependency that severely degrades methods relying on realized-path targets. When LSM estimates the value of exercising



(a) $\ell = 1$ right remaining.



(b) $\ell = 2$ rights remaining.



(c) $\ell = 3$ rights remaining.

Figure 2: Learned exercise boundaries for N=50, L=3, σ=0.6, trained on 2000 paths. The black curve is the exact DP boundary; the shaded region is the optimal exercise region. **LSM** (blue dashed) oscillates wildly, at times triggering exercise deep out-of-the-money and missing the correct boundary by \$30–\$50. **TD-LSM** (orange dashed) tracks the DP boundary closely across all three rights levels. **FQI** and **LSPI** (dotted) converge only in the final few time steps, consistently underestimating the boundary earlier in the option's life.

now, it must account for the fact that the next δ steps yield zero payoff regardless of price movements. With a sparse training set of 100 paths, whether the δ-step waiting window coincides with a favourable price is a matter of chance, so LSM's realized targets are a noisy and potentially misleading signal.

TD-LSM's bootstrapped target side-steps this issue; since $\hat{V}_{t+1}$ is a regressor fit to the value function (which properly integrates over the δ-step waiting period), the target already reflects the expected cost of the constraint. This makes TD-LSM's estimates robust regardless of which particular paths happen to land in the training set. Quantitatively, TD-LSM achieves a payoff of $\approx$24.5 at n=100 paths, rising to $\approx$28 at n=1000, and maintains a lead of 3–8 points over all competitors throughout (Figure 1b).

Two additional failure modes appear in this setting. FQI, which is competitive in other configurations, plateaus at $\approx$22–24: its global Q-function must distinguish states where the agent is free to exercise from states where the δ-step waiting period is still active, but the offline dataset under-represents the last step before these transitions, causing underestimation of the exercise value. LSPI, whose parameter vector must simultaneously model hold and exercise Q-values, degrades from $\approx$21 to $\approx$20 as training data grows, a sign that the LSTDQ update causes states with identical price/rights but different cooldown values to produce near-duplicate feature rows, and hence destabilising the solution.

Across both configurations, TD-LSM's key advantage is that it replaces high-variance realized-path continuation estimates with smooth, bootstrapped value estimates from the already-fitted regressor at the next time step. This single modification yields substantial, consistent gains in mean payoff, both in absolute terms and in sample efficiency, without increasing computational cost relative to LSM.

7. Extension to Deep Reinforcement Learning

While the tabular and linear regression-based methods presented in Section 4 are highly computationally efficient and mathematically tractable for low-dimensional problems, they become increasingly susceptible to the curse of dimensionality as the state space grows. For complex environments—such as pricing basket options with a large number of underlying assets ($M \gg 1$), or environments requiring complex historical

path dependencies—manually engineered polynomial basis functions become inadequate. To handle such high-dimensional, non-linear state spaces, we can naturally extend our multiple-stopping framework by leveraging the universal approximation capabilities of deep neural networks.

Remark 2. The deep reinforcement learning architectures discussed in this section represent an exploratory extension of our offline multiple-stopping framework. Extensive empirical evaluation, rigorous hyperparameter tuning, and large-scale benchmarking remain active areas of ongoing research. Further work is required to definitively characterize their comparative sample efficiency, stability, and computational trade-offs against classical tabular methods in strictly data-scarce offline regimes. The results and discussions we have found thus far are mentioned in Appendix C.4

We briefly outline four deep reinforcement learning approaches adapted for the optimal multiple stopping problem. The full algorithmic pseudocodes for these methods are deferred to Appendix B.

7.1. Deep Backward Induction (Deep LSM and Deep TD-LSM)

Deep LSM and Deep TD-LSM replace the linear ordinary least squares (OLS) regression in the standard backward-induction framework with multi-layer perceptrons (MLPs). At each time step t and rights level k, rather than fitting a polynomial vector, we train two separate neural networks using gradient descent (e.g., via Adam):

- A continuation network $f_{\theta^{\text{cont}}}(s)$ to estimate the expected value of holding.
- A residual network $f_{\theta^{\text{ex}}}(s)$ to estimate the expected value of the $k-1$ remaining rights upon exercising.

Consistent with the tabular approach, training is restricted to in-the-money (ITM) paths to focus the network's capacity on the decision boundary. A critical architectural optimization in this sequence of networks is *warm-starting*: the weights of the networks at time t are initialized using the fully trained weights from time $t+1$. Because the value surface changes continuously over small time increments, warm-starting drastically reduces the required number of training epochs per time step and prevents catastrophic forgetting.

The distinction between the two methods mirrors their tabular counterparts: Deep LSM trains these networks against the high-variance, realized Monte Carlo path returns, whereas Deep TD-LSM trains them against the smooth, one-step bootstrapped estimates generated by the networks at time $t+1$.

7.2. Global Deep Q-Learning (DQN and CQL)

Unlike the backward-induction methods that train hundreds of small, localized networks sequentially, Deep Q-Networks (DQN) and Conservative Q-Learning (CQL) parameterize the entire state-action-time space using a single, global neural network $Q_\theta(s, t, k, a)$.

Deep Q-Networks (DQN): Standard offline DQN trains this global network by sampling mini-batches from a replay buffer containing the fixed dataset of transitions, minimizing the temporal-difference Bellman error. While powerful in continuous control, standard forward-moving DQN struggles with finite-horizon optimal stopping problems; the sparse, terminal nature of exercise payoffs makes it difficult to stably propagate credit backward across long horizons without algorithmic modifications.

Conservative Q-Learning (CQL): In a strictly offline setting, global Q-networks are highly prone to overestimation bias, where the network spuriously hallucinates high values for out-of-distribution (OOD) states or actions not present in the dataset. CQL mitigates this by augmenting the standard Bellman Mean Squared Error (MSE) objective with a conservatism penalty. It explicitly regularizes the Q-values by minimizing the values of unseen actions while maximizing the values of actions actually taken in the dataset, effectively pushing the learned Q-function to be a lower bound on the true value.

8. Conclusion and Future Work

In this work, we bridged the gap between classical optimal stopping theory and modern offline reinforcement learning by formulating optimal multiple stopping problems as augmented-state MDPs. By explicitly tracking remaining exercise rights, we captured the critical value coupling inherent in repeated-sell and buy-sell dynamics. We introduced Bootstrapped Least-Squares Monte Carlo (TD-LSM), demonstrating that substituting high-variance Monte Carlo realized returns with one-step lookahead continuation values dramatically improves sample efficiency and stability in strictly offline, data-scarce regimes. Our empirical evaluations confirmed that TD-LSM outperforms LSM, FQI and LSPI in high variance and low data regimes. Furthermore, our theoretical analysis provided rigorous finite-sample bounds, revealing that multiple stopping inherently incurs an $O(K)$ penalty compared to single stopping.

While our current analysis focuses strictly on the offline learning regime, extending this framework to the online setting represents a critical next step. Developing an online variant of TD-LSM, potentially leveraging random trajectory lengths and active temporal-difference updates, would allow agents to adapt dynamically to non-stationary financial environments. On the theoretical front, we plan to develop rigorous finite-sample performance error bounds for LSM and TD-LSM in the offline multiple stopping setting, as well as further analyze their deep reinforcement learning variations. Additionally, we aim to explore policy gradient methods that explicitly enforce a threshold-type prior on the network architecture, narrowing the hypothesis space to match the theoretical properties of optimal stopping rules. Finally, investigating non-parametric regression techniques, such as Gaussian Processes, could offer superior uncertainty quantification for the continuation value surface in high-dimensional, continuous state spaces.

References

[1] Albright, S. C. (1977). A Bayesian approach to a generalized house selling problem. *Management Science*, 24(4):432–440.

[2] Antos, A., Szepesvári, C., and Munos, R. (2008). Learning near-optimal policies with Bellman-residual minimization based fitted policy iteration and a single sample path. *Machine Learning*, 71:89–129.

[3] Becker, S., Cheridito, P., and Jentzen, A. (2019). Deep optimal stopping. *Journal of Machine Learning Research*, 20(74):1–25.

[4] Bertsekas, D. P. and Tsitsiklis, J. N. (1996). *Neuro-Dynamic Programming*. Athena Scientific.

[5] Carmona, R. and Touzi, N. (2008). Optimal multiple stopping and valuation of swing options. *Mathematical Finance*, 18(2):239–268.

[6] Chow, Y.-S., Robbins, H., and Siegmund, D. (1971). *Great Expectations: The Theory of Optimal Stopping*. Houghton Mifflin, Boston.

[7] Gilbert, J. P. and Mosteller, F. (1966). Recognizing the maximum of a sequence. *Journal of the American Statistical Association*, 61:35–73.

[8] Häggström, G. (1967). Optimal sequential procedures when more than one stop is required. *Annals of Mathematical Statistics*, 38(5):1618–1626.

[9] Huré, C., Pham, H., and Warin, X. (2020). Deep backward schemes for high-dimensional nonlinear PDEs. *Mathematics of Computation*, 89(324):1547–1579.

[10] Kakade, S. and Langford, J. (2002). Approximately optimal approximate reinforcement learning. In *Proceedings of ICML*, 267–274.

[11] Kobylanski, M., Quenez, M.-C., and Rouy-Mironescu, E. (2011). Optimal multiple stopping time problem. *Annals of Applied Probability*, 21(4):1365–1399.

[12] Lagoudakis, M. G. and Parr, R. (2003a). Least-squares policy iteration. In *Proceedings of ICML*.

[13] Lagoudakis, M. G. and Parr, R. (2003b). Least-squares policy iteration. *Journal of Machine Learning Research*, 4:1107–1149.

[14] Lapeyre, B. and Lelong, J. (2019). Neural network regression for Bermudan option pricing. *arXiv preprint arXiv:1907.06474*.

[15] Damera Venkata, N. and Bhattacharyya, C. (2023). Deep recurrent optimal stopping. *Advances in Neural Information Processing Systems*, 36.

[16] Laurière, M. and Talbi, M. (2025). Deep learning for the multiple optimal stopping problem. *arXiv preprint arXiv:2512.22961*.

[17] Li, Y., Szepesvári, C., and Schuurmans, D. (2009). Learning exercise policies for American options. *AISTATS 2009*, JMLR W&CP 5:352–359.

[18] Longstaff, F. A. and Schwartz, E. S. (2001). Valuing American options by simulation: a simple least-squares approach. *Review of Financial Studies*, 14(1):113–147.

[19] Puterman, M. L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.

[20] Rosenfield, D. B., Shapiro, R. D., and Butler, D. A. (1983). Optimal strategies for selling an asset. *Management Science*, 29(9):1051–1061.

[21] Rouillard, J., Warin, X., and Pham, H. (2021). Solving optimal stopping problems with deep Q-learning. *arXiv preprint arXiv:2101.09682*.

[22] Shiryaev, A. N. (1978). *Optimal Stopping Rules*. Springer, New York.

[23] Sofronov, G. Y. (2020). An optimal double stopping rule for a buying-selling problem. *European Journal of Operational Research*, 283(1):126–136.

[24] Stadje, W. (1985). On multiple stopping rules. *Optimization*, 16:401–418.

[25] Stone, C. J. (1982). Optimal global rates of convergence for nonparametric regression. *Annals of Statistics*, 10(4):1040–1053.

[26] Tropp, J. A. (2012). User-friendly tail bounds for sums of random matrices. *Foundations of Computational Mathematics*, 12(4):389–434.

[27] Tsitsiklis, J. N. and Van Roy, B. (1999). Optimal stopping of Markov processes: Hilbert space theory, approximation algorithms, and an application to pricing high-dimensional financial derivatives. *IEEE Transactions on Automatic Control*, 44(10):1840–1851.

[28] Tsitsiklis, J. N. and Van Roy, B. (2001). Regression methods for pricing complex American-style options. *IEEE Transactions on Neural Networks*, 12(4):694–703.

[29] Watkins, C. J. C. H. and Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3):279–292.

[30] Yu, H. and Bertsekas, D. P. (2007). Q-learning algorithms for optimal stopping based on least squares. MIT LIDS Technical Report 2731.

[31] Poor, H. V. and Hadjiliadis, O. (2009). *Quickest Detection*. Cambridge University Press.

[32] Ross, S. M. (1970). *Applied Probability Models with Optimization Applications*. Holden-Day.

[33] Shiryaev, A. N. (2007). *Optimal Stopping Rules* (3rd ed.). Springer.

[34] Sofronov, G. Y. and Szajowski, K. (2024). *Multiple Stopping Problems: Unilateral and Multilateral Approaches*. CRC Press.

[35] Ferguson, T. S. (1989). Who solved the secretary problem? *Statistical Science*, 4(3):282–289.

[36] McCall, J. J. (1970). Economics of information and job search. *The Quarterly Journal of Economics*, 84(1):113–126.

[37] Tartakovsky, A., Nikiforov, I., and Basseville, M. (2014). *Sequential Analysis: Hypothesis Testing and Changepoint Detection*. CRC Press.

[38] R. Carmona and N. Touzi, "Optimal Multiple Stopping and Valuation of Swing Options," *Mathematical Finance*, vol. 18, no. 2, pp. 239–268, 2008. doi: 10.1111/j.1467-9965.2007.00331.x.

[39] A. Mahajan, "Monotone MDPs – Notes on Stochastic Control," 2024. Available: `https://adityam.github.io/stochastic-control/mdps/monotone-mdps.html`

Appendix A. Analysis of Structural Properties

This appendix provides proofs of the structural properties stated in Section 3, explicitly relying on conditions A1-A3.

Proof of Lemma 1 (Structural Properties of the Value Function)

We establish each structural property through backward induction on time $t \in \{0, 1, \dots, T\}$.

1. Monotonicity in Remaining Rights:

Repeated-Sell Setting: We prove $V_t(s, k+1) \geq V_t(s, k)$ for all t, s, and $k \geq 1$.

Base case: At terminal time T, both value functions equal zero: $V_T(s, k+1) = V_T(s, k) = 0$.

Inductive step: Assume $V_{t+1}(s, k+1) \geq V_{t+1}(s, k)$ holds for all states s. Consider the Bellman equations at time t:

$$V_t(s, k+1) = \max\Big\{ \underbrace{g(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s]}_{Q_t(s,k+1,1)}, \underbrace{h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k+1) \mid S_t = s]}_{Q_t(s,k+1,0)} \Big\}$$

$$V_t(s, k) = \max\Big\{ \underbrace{g(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-1) \mid S_t = s]}_{Q_t(s,k,1)}, \underbrace{h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s]}_{Q_t(s,k,0)} \Big\}$$

By the inductive hypothesis, $V_{t+1}(s, k+1) \geq V_{t+1}(s, k)$ for all s, which implies:

$$Q_t(s, k+1, 0) = h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k+1) \mid S_t = s] \geq h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s] = Q_t(s, k, 0)$$

Furthermore, since $g(s) \geq 0$ (by Assumption A1), we have:

$$Q_t(s, k+1, 1) = g(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s] \geq \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s]$$

Now observe that an agent with $k+1$ rights can always replicate any policy available to an agent with k rights by choosing action $a = 0$ (wait) in the first decision period, thereby preserving $k+1$ rights and achieving value $Q_t(s, k+1, 0) \geq Q_t(s, k, 0) \geq V_t(s, k)$. Since the agent with $k+1$ rights can also exercise immediately, we have:

$$V_t(s, k+1) = \max\{Q_t(s, k+1, 1), Q_t(s, k+1, 0)\} \geq Q_t(s, k+1, 0) \geq V_t(s, k)$$

Buy-Sell Setting: We prove $V_t(s, k+2) \geq V_t(s, k)$ for all t, s, and $k \geq 1$.

The argument parallels the repeated-sell case. An agent with $k+2$ rights has exactly one additional buy-sell cycle compared to an agent with k rights. Since individual rights strictly alternate the sign of cash

flow (buy requires paying $g(s)$, sell receives $g(s)$), a state with $k+1$ rights does not subsume the action space of k rights. However, $k+2$ rights provide one complete additional cycle, which the agent can optimally choose to never execute. Therefore, any policy feasible for k rights can be replicated with $k+2$ rights by never entering the additional trade cycle, establishing $V_t(s, k+2) \geq V_t(s, k)$ via backward induction.

2. Time Monotonicity:

We prove $V_t(s, k) \geq V_{t+1}(s, k)$ for all states s and remaining rights k, assuming non-negative holding rewards $h(s) \geq 0$ and discount factor $\gamma \in (0, 1]$.

Base case: The property holds trivially at $t = T - 1$ since $V_T(s, k) = 0$ for all s and k, giving:

$$V_{T-1}(s, k) = \max\{g(s), h(s)\} \geq 0 = V_T(s, k)$$

Inductive step: Assume $V_{t+1}(s, k) \geq V_{t+2}(s, k)$ for all s and k. At time t, the agent can choose to wait (action $a = 0$), yielding:

$$V_t(s, k) \geq Q_t(s, k, 0) = h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s]$$

Since $h(s) \geq 0$, $\gamma \leq 1$, and $V_{t+1}(s, k) \geq 0$ (which follows by induction on the non-negativity of g and h), we need to establish the relationship between $Q_t(s, k, 0)$ and $V_{t+1}(s, k)$.

The key insight is that the waiting strategy at time t dominates starting fresh at time $t+1$ in the same state because: (i) the agent collects the holding reward $h(s) \geq 0$, and (ii) the continuation value is identical. Formally, for any realization $s' = s$ (i.e., the state remains unchanged), we have:

$$Q_t(s', k, 0)\Big|_{s'=s} = h(s) + \gamma V_{t+1}(s, k) \geq V_{t+1}(s, k)$$

More generally, even when states transition stochastically, the expanded time horizon at t compared to $t+1$ provides additional optionality (one extra decision period), ensuring:

$$V_t(s, k) = \max\{Q_t(s, k, 1), Q_t(s, k, 0)\} \geq V_{t+1}(s, k)$$

This holds because the agent at time t can always delay all decisions by one period and then implement the optimal policy from $t+1$ onward, collecting $h(s) \geq 0$ in the interim.

3. Diminishing Marginal Value:

Repeated-Sell Setting: We prove the concavity property $\Delta_t(s, k) \leq \Delta_t(s, k-1)$ for $k \geq 2$, where $\Delta_t(s, k) := V_t(s, k) - V_t(s, k-1)$ denotes the marginal value of the k-th right.

Base case: At terminal time T, all value functions are zero: $V_T(s, k) = 0$ for all k. Thus, $\Delta_T(s, k) = 0 \leq 0 = \Delta_T(s, k-1)$.

Inductive step: Assume $\Delta_{t+1}(s, k) \leq \Delta_{t+1}(s, k-1)$ holds for all states s and $k \geq 2$. We must show this property propagates to time t.

From the Bellman optimality equation:

$$\begin{aligned}\Delta_t(s, k) &= V_t(s, k) - V_t(s, k-1) \\ &= \max\{Q_t(s, k, 1), Q_t(s, k, 0)\} - \max\{Q_t(s, k-1, 1), Q_t(s, k-1, 0)\}\end{aligned}$$

Expanding the Q-functions:

$$\begin{aligned}Q_t(s, k, 1) &= g(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-1) \mid S_t = s] \\ Q_t(s, k, 0) &= h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) \mid S_t = s] \\ Q_t(s, k-1, 1) &= g(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-2) \mid S_t = s] \\ Q_t(s, k-1, 0) &= h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k-1) \mid S_t = s]\end{aligned}$$

Notice that:

$$\begin{aligned}Q_t(s, k, 0) - Q_t(s, k-1, 0) &= \gamma \mathbb{E}[V_{t+1}(S_{t+1}, k) - V_{t+1}(S_{t+1}, k-1) \mid S_t = s] \\ &= \gamma \mathbb{E}[\Delta_{t+1}(S_{t+1}, k) \mid S_t = s]\end{aligned}$$

Similarly:

$$\begin{aligned} Q_t(s,k,1) - Q_t(s,k-1,1) &= \gamma\mathbb{E}[V_{t+1}(S_{t+1},k-1) - V_{t+1}(S_{t+1},k-2) \mid S_t = s] \\ &= \gamma\mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s] \end{aligned}$$

By the inductive hypothesis, $\Delta_{t+1}(s,k) \leq \Delta_{t+1}(s,k-1)$ for all s. Therefore:

$$Q_t(s,k,0) - Q_t(s,k-1,0) = \gamma\mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s] \leq \gamma\mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s] = Q_t(s,k,1) - Q_t(s,k-1,1)$$

This inequality states that the marginal value of waiting (holding the k-th right) is weakly smaller than the marginal value of exercising (converting the k-th right into the $(k-1)$-th state).

To complete the argument, we invoke the following: for any four values $\{a_1, a_2, b_1, b_2\}$ satisfying $a_2 - a_1 \leq b_2 - b_1$, it holds that:

$$\max\{a_2, b_2\} - \max\{a_1, b_1\} \leq b_2 - b_1$$

Applying this with $a_1 = Q_t(s,k-1,0)$, $a_2 = Q_t(s,k,0)$, $b_1 = Q_t(s,k-1,1)$, $b_2 = Q_t(s,k,1)$, we obtain:

$$\Delta_t(s,k) = V_t(s,k) - V_t(s,k-1) \leq Q_t(s,k,1) - Q_t(s,k-1,1) = \gamma\mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s] \leq \Delta_{t+1}(s,k-1)$$

where the final inequality uses Assumption A2 (stochastically monotone transitions) to ensure the expectation preserves the ordering under state transitions.

Buy-Sell Setting: The proof for cycle marginal value $\Delta_t^{\text{cycle}}(s,k) := V_t(s,k) - V_t(s,k-2)$ follows identically. By backward induction and the stochastic monotonicity of transitions, the expected marginal benefit of an additional complete buy-sell cycle is non-increasing: $\Delta_t^{\text{cycle}}(s,k) \leq \Delta_t^{\text{cycle}}(s,k-2)$ for $k \geq 3$. □

Proof of Lemma 2 (Threshold-Type Optimal Policies)

We characterize the optimal policy structure by analyzing the *stopping premium*, defined as the incremental value of exercising versus waiting.

1. Repeated-Sell Formulation:

The optimal policy at time t dictates exercising when $Q_t(s,k,1) \geq Q_t(s,k,0)$. Define the stopping premium:

$$\Delta Q_t(s,k) := Q_t(s,k,1) - Q_t(s,k,0)$$

Substituting the Bellman equations:

$$\begin{aligned} \Delta Q_t(s,k) &= \Big[g(s) + \gamma\mathbb{E}[V_{t+1}(S_{t+1},k-1) \mid S_t = s]\Big] \\ &\quad - \Big[h(s) + \gamma\mathbb{E}[V_{t+1}(S_{t+1},k) \mid S_t = s]\Big] \\ &= g(s) - h(s) - \gamma\mathbb{E}[V_{t+1}(S_{t+1},k) - V_{t+1}(S_{t+1},k-1) \mid S_t = s] \\ &= g(s) - h(s) - \gamma\mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s] \end{aligned}$$

Under Assumption A1, both $g(s)$ and $h(s)$ are weakly increasing in s. Under Assumption A2, the transition kernel is stochastically monotone, meaning higher current states lead to higher expected future states. Since $g(s) - h(s)$ is weakly increasing and $\Delta_{t+1}(s,k)$ is independent of the exercise decision (the state evolution does not depend on stopping actions), Assumption A3 guarantees that $\Delta Q_t(s,k)$ is supermodular—that is, weakly increasing in s.

Therefore, there exists a threshold θ_t^k such that:

$$\pi_t^*(s,k) = \begin{cases} 1 & \text{if } s \geq \theta_t^k \\ 0 & \text{if } s < \theta_t^k \end{cases}$$

where θ_t^k is defined implicitly by $\Delta Q_t(\theta_t^k, k) = 0$ (with appropriate conventions at boundary cases).

From Lemma 1, we have the diminishing marginal value property: $\Delta_t(s,k) \leq \Delta_t(s,k-1)$ for all s and $k \geq 2$. This implies:

$$\mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s] \leq \mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s]$$

Substituting into the stopping premium:

$$\Delta Q_t(s,k) = g(s) - h(s) - \gamma \mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s]$$
$$\Delta Q_t(s,k-1) = g(s) - h(s) - \gamma \mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s]$$

Since $\mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s] \leq \mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s]$, we have:

$$-\gamma \mathbb{E}[\Delta_{t+1}(S_{t+1},k) \mid S_t = s] \geq -\gamma \mathbb{E}[\Delta_{t+1}(S_{t+1},k-1) \mid S_t = s]$$

Therefore:

$$\Delta Q_t(s,k) \geq \Delta Q_t(s,k-1) \quad \text{for all } s$$

This inequality has a critical implication: if $\Delta Q_t(s,k)$ is weakly larger than $\Delta Q_t(s,k-1)$ for all states, and both functions are weakly increasing in s (by supermodularity), then $\Delta Q_t(s,k)$ crosses zero at a weakly lower state value than $\Delta Q_t(s,k-1)$. Formally:

$$\theta_t^k = \inf\{s : \Delta Q_t(s,k) \geq 0\} \leq \inf\{s : \Delta Q_t(s,k-1) \geq 0\} = \theta_t^{k-1}$$

This establishes the nested threshold structure: agents with more remaining rights have lower exercise thresholds, meaning they exercise more aggressively (over a larger region of the state space).

2. Buy-Sell Formulation:

For buying (even k), the agent pays $g(s)$ to acquire the asset. The stopping premium for buying is:

$$\Delta Q_t^{\text{buy}}(s,k) := Q_t(s,k,1) - Q_t(s,k,0) = -g(s) + h(s) + \gamma \mathbb{E}[V_{t+1}(S_{t+1},k-1) - V_{t+1}(S_{t+1},k) \mid S_t = s]$$

Under Assumption A3, when the reward for buying is $-g(s)$ (a cost), the stopping premium is submodular—that is, weakly decreasing in s. Therefore, there exists an upper threshold θ_t^{buy} such that buying is optimal when $s \leq \theta_t^{\text{buy}}$.

For selling (odd $k-1$), the agent receives $g(s)$ from selling the asset. The stopping premium for selling is:

$$\Delta Q_t^{\text{sell}}(s,k-1) := Q_t(s,k-1,1) - Q_t(s,k-1,0) = g(s) - h(s) - \gamma \mathbb{E}[V_{t+1}(S_{t+1},k-1) - V_{t+1}(S_{t+1},k-2) \mid S_t = s]$$

Under Assumption A3, the selling premium is supermodular—weakly increasing in s. Therefore, there exists a lower threshold θ_t^{sell} such that selling is optimal when $s \geq \theta_t^{\text{sell}}$.

To justify entering a complete buy-sell trade cycle, the expected profit from buying at state s_{buy} and subsequently selling at state s_{sell} must be non-negative. Consider an agent who buys at the marginal buy threshold $s = \theta_t^{\text{buy}}$ (where $\Delta Q_t^{\text{buy}}(\theta_t^{\text{buy}},k) = 0$) and sells at the marginal sell threshold $s = \theta_t^{\text{sell}}$ (where $\Delta Q_t^{\text{sell}}(\theta_t^{\text{sell}},k-1) = 0$).

At the buy threshold:

$$-g(\theta_t^{\text{buy}}) + h(\theta_t^{\text{buy}}) + \gamma \mathbb{E}[V_{t+1}(S_{t+1},k-1) - V_{t+1}(S_{t+1},k) \mid S_t = \theta_t^{\text{buy}}] = 0$$

At the sell threshold (conditional on already holding the asset):

$$g(\theta_t^{\text{sell}}) - h(\theta_t^{\text{sell}}) - \gamma \mathbb{E}[V_{t+1}(S_{t+1},k-1) - V_{t+1}(S_{t+1},k-2) \mid S_t = \theta_t^{\text{sell}}] = 0$$

For a profitable trade cycle, the selling price must exceed the buying price. Since $g(s)$ is weakly increasing (Assumption A1), and selling requires $g(s)$ to be sufficiently high relative to the opportunity cost while buying requires $g(s)$ to be sufficiently low, we must have:

$$\theta_t^{\text{buy}} \leq \theta_t^{\text{sell}}$$

This hysteresis band ensures that the agent does not buy and immediately sell (or vice versa) at the same state, which would result in instantaneous losses due to transaction frictions embedded in the holding cost structure. The separation between buy and sell thresholds reflects the minimum price spread required to justify entering and subsequently exiting a position. □

Remark 3. The assumptions A1-A3 apply in the environment we have for our experiments in Appendix C.1 in the following ways:

- **(A1) Monotone Rewards:** The immediate reward upon exercising a right is $r(s,k,1) = \max(K - s, 0)$, which is trivially weakly decreasing in s (equivalently, weakly increasing in $-s$, preserving the monotonicity properties under sign reversal).
- **(A2) Stochastically Monotone Transitions:** We model the underlying asset dynamics using Geometric Brownian Motion (GBM). The transition dynamic is given by $S_{t+1} = S_t \exp\left((\mu - \frac{\sigma^2}{2})\Delta t + \sigma\sqrt{\Delta t}Z\right)$. Because the exponential factor is strictly positive and independent of S_t, if $s_1 \geq s_2$, then $s_1 \exp(\dots) \geq s_2 \exp(\dots)$ almost surely for any realization of the standard normal variable Z. Consequently, the transition kernel $\mathcal{P}$ pathwise preserves the ordering of states, meaning **A2** is strictly satisfied.
- **(A3) Modularity of the Q-function:** In our formulation, the underlying GBM price transitions are exogenous to the agent's actions. Because **A2** holds (stochastically monotone transitions) and **A1** holds (monotonic immediate rewards), standard MDP theory guarantees that the continuation value $\mathbb{E}[V_{t+1}(S_{t+1}, k')]$ preserves the monotonicity of the value function. Consequently, the stopping premium $\Delta Q_t(s,k) = r(s,k,1) - \mathbb{E}[V_{t+1}(S_{t+1}, k')]$ is guaranteed to be monotonic in s, satisfying **A3** and ensuring the optimality of a threshold-based policy.

Appendix A.1. Empirical Verification of Structural Properties

To empirically verify the structural properties stated in Lemmas 1 and 2, we compute the exact value function and optimal policy via backward induction on a fine discretization of the state space. Concretely, we model a put option under GBM with $S_0 = K = 100$, volatility $\sigma = 0.2$, risk-free rate $r = 0.05$, maturity $T = 1$ year, $N = 50$ exercise dates, $L = 3$ exercise rights, and no mandatory waiting period. The price space is discretized into a uniform log-price grid of 400 points spanning $[0.10K,\ 3.00K]$; the transition matrix is derived analytically from the log-normal GBM density. Backward induction is run per rights level, yielding exact value functions $V_t(s,k)$ and optimal decisions for every (t,s,k) triple.

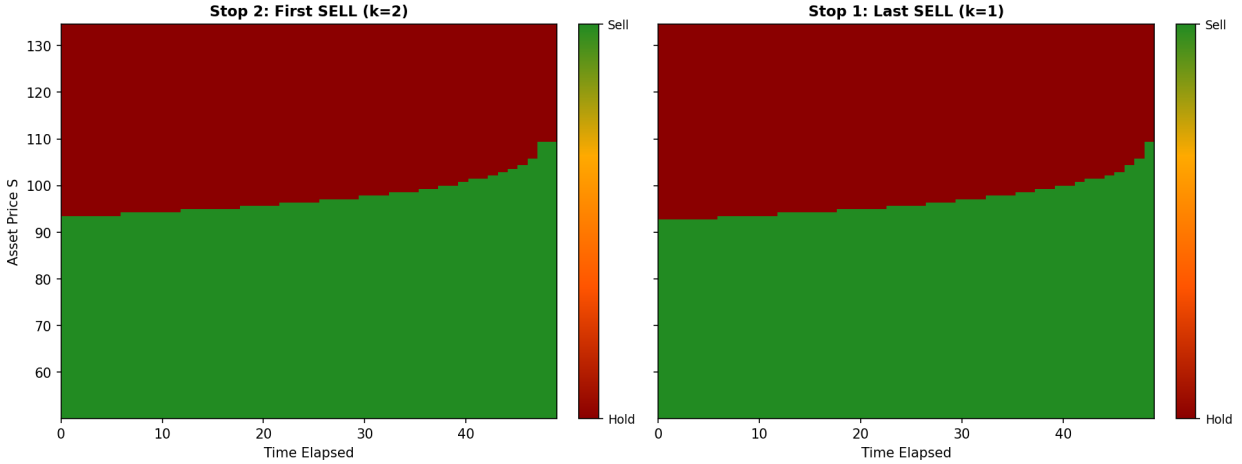


Figure A.3: Optimal exercise policy over time and asset price. **Left** ($k = 2$, first sell): the agent is conservative, holding a second right in reserve, resulting in a smaller exercise region. **Right** ($k = 1$, last sell): the agent exercises over a larger region as no rights remain. The rightward boundary shift reflects the time-value effect. The nested structure $\theta_t^{k=2} \leq \theta_t^{k=1}$ (Lemma 2) is evident.

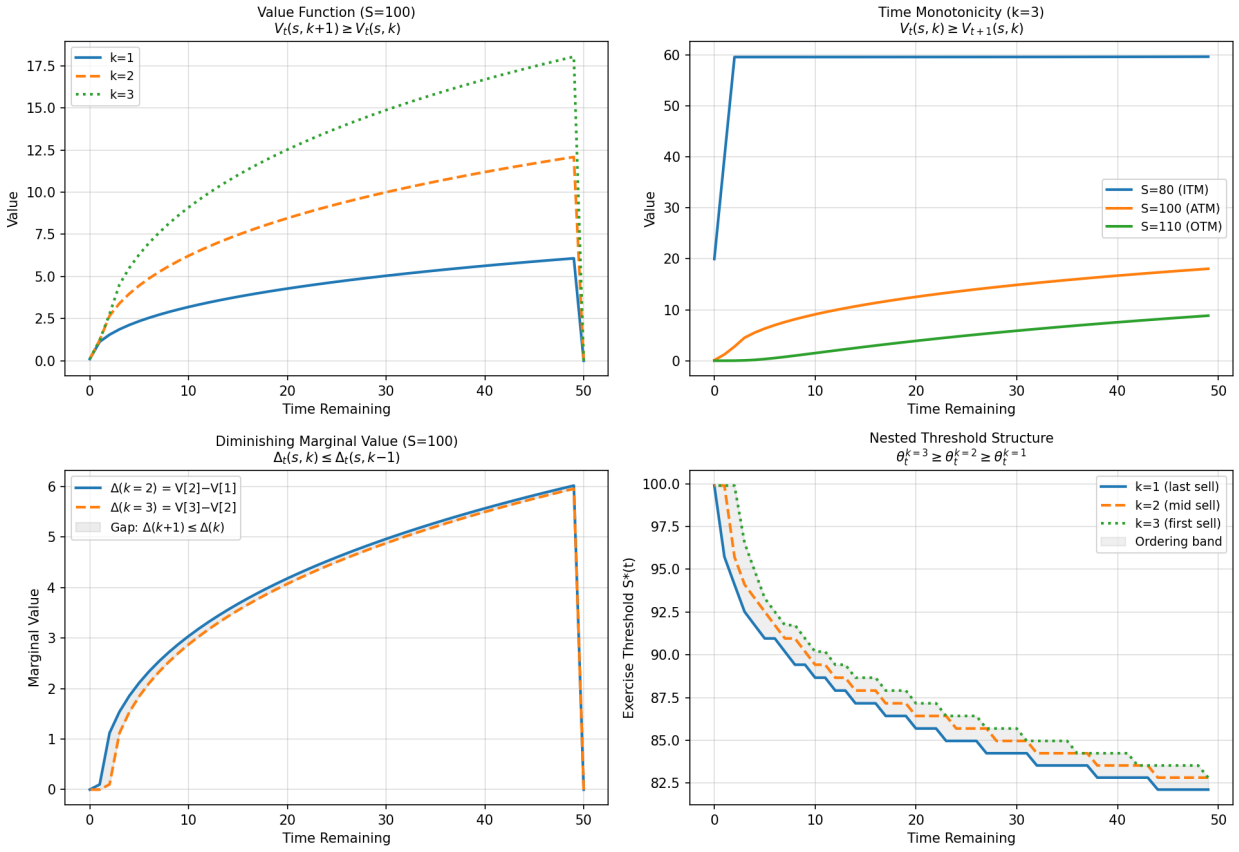


Figure A.4: Structural properties of the value function $V_t(s,k)$ under GBM. **Top-left:** Monotonicity in exercise rights: $V_t(s,3) > V_t(s,2) > V_t(s,1)$ at $s = K$ (Lemma 1, prop. 1). **Top-right:** Time monotonicity: curves for $k = 3$ increase as time remaining grows (prop. 2). **Bottom-left:** Diminishing marginal value: $\Delta(k=3) \leq \Delta(k=2)$ at $s = K$ (prop. 3). **Bottom-right:** Nested thresholds: $\theta_t^{k=3} \geq \theta_t^{k=2} \geq \theta_t^{k=1}$, confirming more rights lead to more aggressive exercise (Lemma 2).

Appendix B. Algorithm Pseudo-codes

Algorithm 3 Fitted Q-Iteration (FQI) for Optimal Multiple Stopping

Hyperparameters: Spatio-temporal basis $\phi(s,t) \in \mathbb{R}^7$, Discount factor γ, Max iterations M, Tolerance ϵ
Input: Set of N paths $\{s_{n,t}\}_{n=1}^{N}$ for $t = 1, \ldots, T$.
Initialize: Policy dictionary $\mathcal{W} = \{0 : \mathbf{0} \in \mathbb{R}^7\}$.
Precompute Global Design Matrices:
Stack features for all paths and times $t \in [1, T-1]$: $X \leftarrow \text{Stack}(\phi(s_{n,t}, t)) \in \mathbb{R}^{N(T-1)\times 7}$
Stack features for next steps $t+1$: $X' \leftarrow \text{Stack}(\phi(s_{n,t+1}, t+1))$ ▷ X' rows are $\mathbf{0}$ if $t+1 = T$
Stack immediate next payoffs: $R' \leftarrow \text{Stack}(r(s_{n,t+1}, 1))$
Precompute Constant Inverse:
$A \leftarrow X^\top X$
$A_{\text{inv}} \leftarrow (A^\top A)^{-1} A^\top$ ▷ Precompute pseudo-inverse once
for $k = 1$ **to** K **do**
    Initialize weights $w_k \leftarrow \mathbf{0} \in \mathbb{R}^7$
    Residual vector $Q_{\text{residual}} \leftarrow X'\mathcal{W}[k-1]$ ▷ Precompute lower-layer continuation
    Total exercise payoff $P_{\text{ex}} \leftarrow R' + Q_{\text{residual}}$
    for $i = 1$ **to** M **do**
        $Q_{\text{cont}} \leftarrow X' w_k$ ▷ Evaluate current policy continuation
        $V_{\text{next}} \leftarrow \max(P_{\text{ex}},\ Q_{\text{cont}})$ ▷ Element-wise Bellman maximum
        $b \leftarrow \gamma X^\top V_{\text{next}}$
        $w_{\text{new}} \leftarrow A_{\text{inv}} b$ ▷ Global OLS update
        if $\|w_{\text{new}} - w_k\|_2 < \epsilon$ **then**
            $w_k \leftarrow w_{\text{new}}$
            break
        end if
        $w_k \leftarrow w_{\text{new}}$
    end for
    $\mathcal{W}[k] \leftarrow w_k$
end for
Return $\mathcal{W}$

Algorithm 4 Least-Squares Policy Iteration (LSPI) for Optimal Multiple Stopping

Hyperparameters: Spatio-temporal basis $\phi(s,t) \in \mathbb{R}^7$, Discount factor γ, Max iterations M, Tolerance ϵ, Ridge penalty λ
Input: Set of N paths $\{s_{n,t}\}_{n=1}^{N}$ for $t = 1, \ldots, T$.
Initialize: Policy dictionary $\mathcal{W} = \{0 : \mathbf{0} \in \mathbb{R}^7\}$.
Precompute Global Design Matrices:
Stack features: $X \leftarrow \text{Stack}(\phi(s_{n,t}, t))$, $X' \leftarrow \text{Stack}(\phi(s_{n,t+1}, t+1))$ ▷ X' rows are $\mathbf{0}$ if $t+1 = T$
Stack immediate next payoffs: $R' \leftarrow \text{Stack}(r(s_{n,t+1}, 1))$
for $k = 1$ **to** K **do**
    Initialize weights $w_k \leftarrow \mathbf{0} \in \mathbb{R}^7$
    Total exercise payoff $P_{\text{ex}} \leftarrow R' + X'\mathcal{W}[k-1]$
    for $i = 1$ **to** M **do**
        Policy Evaluation:
        $Q_{\text{cont}} \leftarrow X' w_k$
        Boolean mask: $\mathcal{I}_{\text{ex}} \leftarrow P_{\text{ex}} > Q_{\text{cont}}$ ▷ Greedy action selection
        Force $\mathcal{I}_{\text{ex}} \leftarrow$ True for rows where $t+1 = T$ **and** $R' > 0$ ▷ Terminal auto-exercise
        LSTD(0) Matrix Construction:
        $X'_{\text{masked}} \leftarrow X'$
        Zero out rows in X'_{masked} where $\mathcal{I}_{\text{ex}}$ is True ▷ Features vanish if exercise occurs
        $A \leftarrow X^\top (X - \gamma X'_{\text{masked}})$
        $b \leftarrow \gamma X^\top (P_{\text{ex}} \odot \mathcal{I}_{\text{ex}})$ ▷ Element-wise multiply by Boolean mask
        Policy Update:
        $w_{\text{new}} \leftarrow (A + \lambda I)^{-1} b$ ▷ Solve regularized linear system
        if $\|w_{\text{new}} - w_k\|_2 < \epsilon$ **then**
            $w_k \leftarrow w_{\text{new}}$
            break
        end if
        $w_k \leftarrow w_{\text{new}}$
    end for
    $\mathcal{W}[k] \leftarrow w_k$
end for
Return $\mathcal{W}$

Algorithm 5 Deep Longstaff–Schwartz Method (Deep LSM)

Hyperparameters: Neural network architecture f, Learning rate η, Epochs M
Input: Set of N paths $\{s_{n,t}\}_{n=1}^{N}$
Initialize: Value matrix $V \in \mathbb{R}^{N\times(K+1)}$ with terminal payoffs $V_{n,T}(k) = \text{Payoff}(s_{n,T}, k)$.
for $k = 1$ **to** K **do**
  for $t = T-1$ **down to** 0 **do**
    Let $\mathcal{I}_{\text{ITM}}$ be the set of paths n where $\text{Payoff}(s_{n,t}, k) > 0$.
    Define realized targets $Y_n^{\text{cont}} = \gamma V_{n,t+1}(k)$ and $Y_n^{\text{ex}} = \gamma V_{n,t+1}(k-1)$.
    Warm Start Networks: Initialize $\theta_{t,k}^{\text{cont}} \leftarrow \theta_{t+1,k}^{\text{cont}}$ and $\theta_{t,k}^{\text{ex}} \leftarrow \theta_{t+1,k}^{\text{ex}}$.
    Training Step:
    for epoch $= 1$ **to** M **do**
      Sample mini-batch $\mathcal{B} \subset \mathcal{I}_{\text{ITM}}$.
      Update $\theta_{t,k}^{\text{cont}}$ via gradient descent on MSE loss w.r.t Y^{cont}.
      Update $\theta_{t,k}^{\text{ex}}$ via gradient descent on MSE loss w.r.t Y^{ex} (if $k > 1$).
    end for
    Decision & Update Step:
    for $n = 1$ **to** N **do**
      if $n \in \mathcal{I}_{\text{ITM}}$ **then**
        Expected Hold $\leftarrow f_{\theta_{t,k}^{\text{cont}}}(s_{n,t})$
        Expected Exercise $\leftarrow \text{Payoff}(s_{n,t}, k) + f_{\theta_{t,k}^{\text{ex}}}(s_{n,t})$
        if Expected Exercise $>$ Expected Hold **then**
          $V_{n,t}(k) \leftarrow \text{Payoff}(s_{n,t}, k) + Y_n^{\text{ex}}$ ▷ Realized exercise
        else
          $V_{n,t}(k) \leftarrow Y_n^{\text{cont}}$ ▷ Realized continuation
        end if
      else
        $V_{n,t}(k) \leftarrow Y_n^{\text{cont}}$ ▷ OTM paths cannot exercise
      end if
    end for
  end for
end for

Algorithm 6 Deep Bootstrapped LSM (Deep TD-LSM)

Hyperparameters: Neural network architecture f, Learning rate η, Epochs M
Input: Set of N paths $\{s_{n,t}\}_{n=1}^{N}$
Initialize: $\widehat{V}_T(s_{n,T}, k) = \text{Payoff}(s_{n,T}, k)$ for all n, k.
for $k = 1$ **to** K **do**
    for $t = T - 1$ **down to** 0 **do**
        Let $\mathcal{I}_{\text{ITM}}$ be the set of paths n where $\text{Payoff}(s_{n,t}, k) > 0$.
        Define bootstrap targets $y_{n,t}^{\text{cont}} = \gamma \widehat{V}_{t+1}(s_{n,t+1}, k)$ and $y_{n,t}^{\text{ex}} = \gamma \widehat{V}_{t+1}(s_{n,t+1}, k-1)$.
        Warm Start Networks: Initialize $\theta_{t,k}^{\text{cont}} \leftarrow \theta_{t+1,k}^{\text{cont}}$ and $\theta_{t,k}^{\text{ex}} \leftarrow \theta_{t+1,k}^{\text{ex}}$.
        Training Step:
        for epoch $= 1$ **to** M **do**
            Sample mini-batch $\mathcal{B} \subset \mathcal{I}_{\text{ITM}}$.
            Update $\theta_{t,k}^{\text{cont}}$ via gradient descent on MSE loss w.r.t y^{cont}.
            Update $\theta_{t,k}^{\text{ex}}$ via gradient descent on MSE loss w.r.t y^{ex} (if $k > 1$).
        end for
        Value Function Update Step:
        for $n = 1$ **to** N **do**
            if $n \in \mathcal{I}_{\text{ITM}}$ **then**
                $\widehat{Q}^{\text{hold}} \leftarrow f_{\theta_{t,k}^{\text{cont}}}(s_{n,t})$
                $\widehat{Q}^{\text{ex}} \leftarrow \text{Payoff}(s_{n,t}, k) + f_{\theta_{t,k}^{\text{ex}}}(s_{n,t})$
                $\widehat{V}_t(s_{n,t}, k) \leftarrow \max\{\widehat{Q}^{\text{hold}},\ \widehat{Q}^{\text{ex}}\}$
            else
                $\widehat{V}_t(s_{n,t}, k) \leftarrow y_{n,t}^{\text{cont}}$ ▷ OTM paths inherit continuation value directly
            end if
        end for
    end for
end for

Algorithm 7 Conservative Q-Learning (CQL)

Hyperparameters: Q-network Q_θ, Learning rate η, CQL penalty α, Epochs N_{epochs}, Batch size B, Target update frequency C, Discount γ
Input: Offline dataset of transitions $\mathcal{D} = \{(s_i, a_i, r_i, s'_i, d_i)\}_{i=1}^{N}$, where $d_i \in \{0, 1\}$ is the terminal flag.
Initialize: Network weights θ, and target network weights $\theta_{\text{target}} \leftarrow \theta$
for epoch $= 1$ **to** N_{epochs} **do**
    Shuffle $\mathcal{D}$ and partition into mini-batches.
    for each mini-batch $\mathcal{B} \subset \mathcal{D}$ **do**
        Compute Target Values:
        for $i \in \mathcal{B}$ **do**
            if $d_i = 1$ **then**
                $y_i \leftarrow r_i$
            else
                $y_i \leftarrow r_i + \gamma \max_{a' \in \mathcal{A}} Q_{\theta_{\text{target}}}(s'_i, a')$
            end if
        end for
        Compute Losses:
        Standard Bellman Error (MSE):

$$\mathcal{L}_{\text{MSE}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} (y_i - Q_\theta(s_i, a_i))^2$$

        Discrete CQL Penalty (minimize overestimation):

$$\mathcal{L}_{\text{CQL}}(\theta) = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \left(\log \sum_{a' \in \mathcal{A}} \exp\Big(Q_\theta(s_i, a')\Big) - Q_\theta(s_i, a_i) \right)$$

        Total Loss:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{MSE}}(\theta) + \alpha \mathcal{L}_{\text{CQL}}(\theta)$$

        Update Weights:
        Compute gradient $\nabla_\theta \mathcal{L}(\theta)$ and clip to maximum norm of 1.0.
        $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}(\theta)$
    end for
    Target Network Update:
    if epoch $(\text{mod } C) = 0$ **then**
        $\theta_{\text{target}} \leftarrow \theta$
    end if
end for

Appendix C. Experimental Setup and Empirical Results

In this appendix we detail the implementation of the option pricing environment used to evaluate the four tabular offline algorithms. We specify the experimental configurations and present empirical results comparing TD-LSM against LSM, FQI, and LSPI across payoff quality, boundary accuracy, and computational runtime.

Appendix C.1. Option Pricing Environment Dynamics

To systematically benchmark the algorithms, we developed a custom Gymnasium environment that models the optimal multiple stopping problem. To ensure a strictly offline evaluation, the environment is initialized with a fixed dataset of pre-generated Geometric Brownian Motion (GBM) trajectories. All algorithms are trained and evaluated on identical paths.

The environment simulates M independent assets over a finite horizon T, divided into N discrete exercise opportunities. The agent begins with L exercise rights.

State Space:. At any discrete time step $t \in \{1, \ldots, N\}$, the state formulation must encapsulate the asset prices, the time remaining, and the exercise rights remaining. To stabilize function approximation, the observation vector is normalized:

$$s_t = \left[\frac{S_{t,1}}{K}, \frac{S_{t,2}}{K}, \ldots, \frac{S_{t,M}}{K}, \frac{N-t}{N}, \frac{l_t}{L} \right]^\top \in \mathbb{R}^{M+2},$$

where $S_{t,m}$ is the price of the m-th asset at time t, K is the strike price, and $l_t \in \{0, \ldots, L\}$ is the number of remaining exercise rights. Time is encoded as *time remaining* $(N-t)/N$; this encoding is more natural for the exercise boundary, which is a monotone function of time-to-maturity. When a mandatory waiting period of c steps is in effect after each exercise, the current waiting counter $\kappa_t \in \{0, \ldots, c\}$ is appended as a fourth component, normalized by c.

Action Space and Waiting Periods:. The action space is discrete: $\mathcal{A} = \{0, 1\}$, where 0 denotes holding and 1 denotes exercising one right. The environment supports a post-exercise mandatory waiting period of c steps, during which the exercise action is overridden to 0. If the agent exercises when $l_t = 0$ or during the waiting period, the action is masked to hold.

Reward Function:. The environment is parameterized for a generalized basket put (an option whose payoff depends on the average price across M underlying assets rather than any individual asset price). If the agent takes action $a_t = 1$ (and is permitted to do so), the immediate reward is the discounted intrinsic value:

$$r(s_t, 1) = e^{-rt\Delta t} \max\left(K - \frac{1}{M} \sum_{m=1}^{M} S_{t,m},\ 0 \right),$$

where r is the risk-free rate and $\Delta t = T/N$. The reward for holding ($a_t = 0$) is 0.

Terminal Conditions:. An episode terminates when either time expires ($t > N$) or all rights are exhausted ($l_t = 0$). To mimic rational behaviour, if $l_N > 0$ at the terminal step N and the option is in-the-money, the environment forces an auto-exercise of exactly one right.

Appendix C.2. Hyperparameters and Experimental Configurations

For all experiments, the underlying asset prices follow independent Geometric Brownian Motions with initial price $S_0 = 100.0$. The strike price is $K = 100.0$, the time to maturity is $T = 1.0$ year, and the risk-free interest rate is $r = 0.05$.

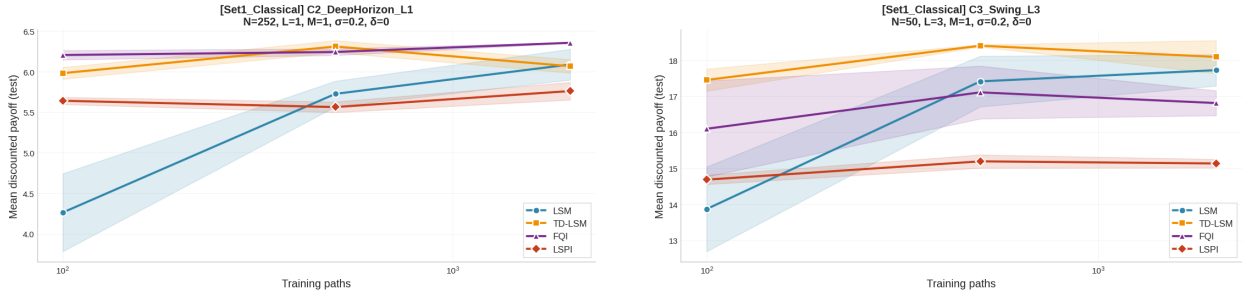
We evaluate the algorithms across four configurations of increasing complexity, varying the number of exercise dates (N), the number of stopping rights (L), asset volatility (σ), and the presence of a mandatory waiting period after each exercise:

1. **Deep-Horizon American Put:** $N = 252$, $L = 1$, $\sigma = 0.2$, $\delta = 0$
2. **Standard Swing Option:** $N = 50$, $L = 3$, $\sigma = 0.2$, $\delta = 0$
3. **High-Volatility Swing Option:** $N = 50$, $L = 3$, $\sigma = 0.6$, $\delta = 0$
4. **Refractory Swing Option:** $N = 100$, $L = 3$, $\sigma = 0.3$, $\delta = 5$

To rigorously test sample efficiency in the offline regime, the algorithms are trained on dataset sizes $|\mathcal{D}_{\text{train}}| \in \{100, 500, 2000, 30000\}$ paths. All policies are subsequently evaluated on a fixed holdout set of 10,000 unseen test paths, averaged over five independent seeds (shaded bands denote ± 1 s.d.).

Appendix C.3. Tabular Results and Discussion:

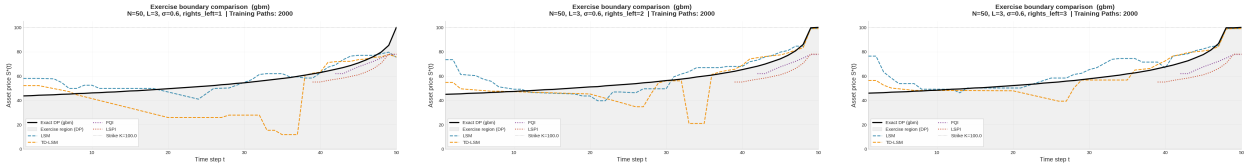
The empirical results reveal a clear and consistent contrast in sample efficiency, and computational cost across the four tabular algorithms. Figures C.5 and **??** report mean discounted test payoffs as a function of training-set size. Figure C.6 shows learned exercise boundaries for representative configurations. Figure C.7 reports training runtimes.



(a) **Deep-Horizon American Put** (N=252, L=1, σ=0.2, no waiting period). LSM's wide confidence bands at small data sizes reflect the compounding of regression errors across 252 backward steps. TD-LSM and FQI remain stable throughout; TD-LSM achieves the highest payoff at n=100 paths.

(b) **Standard Swing Option** (N=50, L=3, σ=0.2, no waiting period). Introducing three exercise rights creates a clear separation. TD-LSM achieves ≈17.5 at n=100 paths, a level that LSM requires n=3000 paths to match. LSPI stagnates near 15.1 regardless of data volume.

Figure C.5: Mean discounted test payoff vs. number of training paths for the Deep-Horizon American Put and the Standard Swing Option. Shaded bands: ±1 s.d. over five seeds. x-axis is log-scaled.



(a) High-Volatility Swing, $\ell = 1$ right.

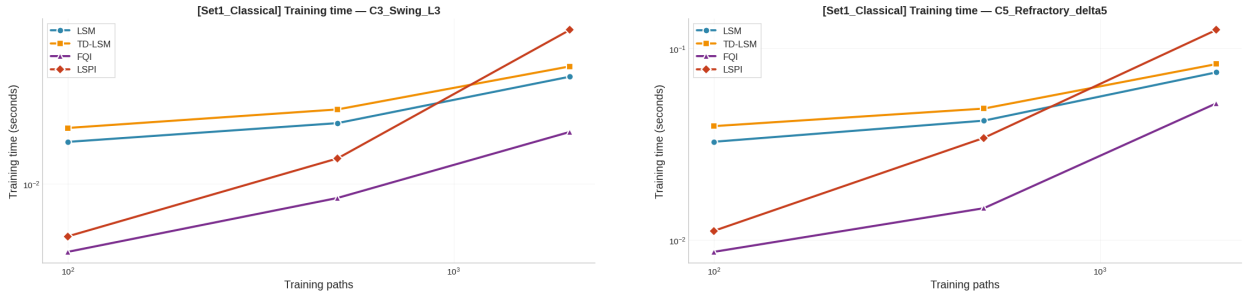
(b) High-Volatility Swing, $\ell = 2$ rights.

(c) High-Volatility Swing, $\ell = 3$ rights.

Figure C.6: Learned exercise boundaries for the High-Volatility Swing Option (N=50, L=3, σ=0.6, trained on 2000 paths). The black curve is the exact DP boundary; the shaded region is the optimal exercise region. **LSM** (blue dashed) oscillates wildly across all three rights levels, at times collapsing \$30–\$50 below the true boundary and triggering spurious early exercise. **TD-LSM** (orange dashed) tracks the DP boundary faithfully at all three rights levels. **FQI** and **LSPI** (dotted) converge to the boundary only near expiry, systematically under-exercising early in the option's life.

Across all tested configurations, TD-LSM consistently performed better than LSM. mean discounted payoff than LSM. For simpler problem settings (shorter horizon, single right), LSM and TD-LSM are broadly competitive. As complexity increases, longer time horizons, more exercise rights, higher volatility, or structural constraints, TD-LSM gains advantage with its bootstrapped target. This is moderate in the standard swing setting (where volatility is low and confidence bands across methods are narrow) and dramatic in the high-volatility setting: at $\sigma = 0.6$, realized continuation values carry enormous variance because a path's outcome from step t to expiry depends on hundreds of large, noisy increments. Bootstrapping from the smooth regressor surface suppresses this noise, producing a stable exercise boundary (Figure C.6) and a persistent ≈4–9 point payoff advantage over LSM. TD-LSM additionally applies its value update across *all* paths (not only in-the-money paths), filling in the out-of-the-money region of the state space where many paths spend significant time at high volatility.

The deep-horizon setting ($N = 252$, single right) exposes a second advantage of TD-LSM: robustness to error compounding in backward induction. With 252 regression steps, a poor fit at any single time step propagates backward into all earlier estimates. LSM's wide confidence bands at small data sizes (Figure C.5a) confirm this: the mean payoff can vary by more than a full point across seeds at n=100 paths, reflecting localized fitting failures that contaminate the entire backward sweep. TD-LSM rebuilds $\hat{V}_t$ from the local



(a) Standard Swing Option (N=50, L=3).

(b) Refractory Swing Option (N=100, L=3).

Figure C.7: Training runtimes (log-scale, seconds) for two representative configurations. LSM and TD-LSM maintain sub-second runtimes across all training-set sizes. LSPI scales worst, and its runtime exceeds 10^{-1} seconds at n=3000 paths even for moderately sized problems. FQI is faster than LSPI but still significantly slower than both LSM variants.

regressor at each step rather than inheriting the raw realized path outcome, limiting the propagation of any single localized error.

The refractory setting exposes failure modes in both FQI and LSPI that do not appear in the other configurations. When an exercise triggers a 5-step mandatory waiting period, the value of exercising now depends critically on what the asset price does over the next five steps, a long-range dependency that is difficult to capture from sparse realized paths. LSM's realized targets are particularly noisy here: whether the waiting window coincides with a favorable price excursion is largely a matter of chance in a small training set. TD-LSM's bootstrapped target already encodes the expected cost of the waiting period through the fitted regressor, making its estimates stable regardless of which paths happen to be sampled.

FQI plateaus despite receiving more data because transitions immediately preceding the waiting period are structurally underrepresented in the offline dataset under a random behavior policy, leaving a systematic gap in Q-function coverage near the exercise boundary. LSPI degrades as data grows: states that differ only in their waiting counter produce nearly identical polynomial feature vectors, progressively worsening the condition number of the LSTDQ normal equations and destabilizing the solution, the only algorithm in our study whose performance falls with additional training data.

Figure C.7 confirms that LSM and TD-LSM train in under one second across all settings and training sizes, making them orders of magnitude faster than LSPI and significantly faster than FQI. The additional computational cost of TD-LSM relative to LSM is negligible: the only extra step is a full-dataset evaluation of the fitted regressor at each time step, which amounts to a single matrix–vector multiply. This combination of sub-second execution and superior payoff makes TD-LSM highly practical for high-frequency or computationally bounded pricing environments.

Appendix C.4. Deep Reinforcement Learning Setting

We now evaluate the four methods in the deep setting, where all algorithms replace polynomial basis functions with neural networks. LSM-Net and TDLSM-Net use the same backward-induction framework as their tabular counterparts but fit small multi-layer perceptrons at each time step. DQN and CQL learn a global Q-function via gradient-based offline training on a collected dataset of transitions. For more details, refer to **??**.

With 252 daily steps and five exercise rights, the algorithm must look into five independent stopping decisions over a year-long horizon. The bootstrapping error that affects TD-based methods accumulates over 252 backward sweeps; for global Q-networks (DQN, CQL), the problem is even worse: the agent must simultaneously learn when to exercise right ℓ given that rights $\ell-1, \ldots, 1$ will be exercised optimally in the future.

TDLSM-Net and LSM-Net achieve payoffs in the range 43–47, while DQN and CQL plateau at 15–17. This gap widens as training data grows because the Q-networks overfit to the high-dimensional transition distribution, whereas TDLSM-Net benefits directly from additional paths through more stable regression

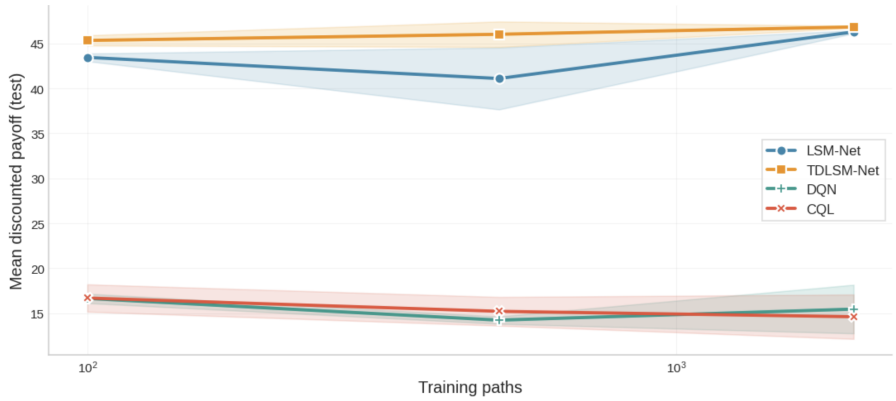
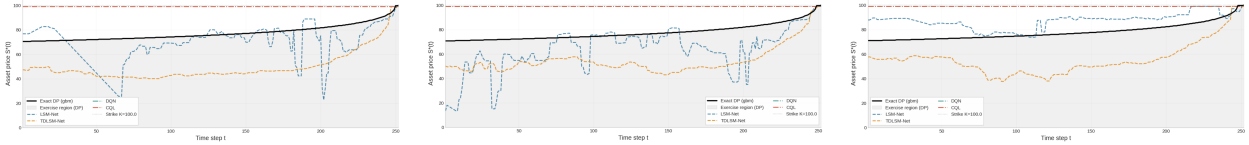


Figure C.8: Mean discounted payoff (test set) vs. number of training paths for the **Daily Swing Option** (N=252, L=5, M=1, σ=0.3). The x-axis is log-scaled; shaded bands are ± 1 standard deviation.



(a) ℓ=3 rights remaining. TDLSM-Net (orange) tracks below the exact boundary, exercising conservatively; LSM-Net (blue) oscillates; DQN/CQL sit at the strike.

(b) ℓ=4 rights remaining. TDLSM-Net tightens around the true boundary; LSM-Net boundary collapses intermittently to deep in-the-money levels; DQN/CQL remain near-flat.

(c) ℓ=5 rights remaining (first exercise). LSM-Net (blue) substantially over-exercises, sitting above the exact boundary; TDLSM-Net (orange) tracks below, consistent with option-value preservation across all five rights.

Figure C.9: Exercise boundaries for the Daily Swing Option (N=252, L=5, σ=0.3, 2000 training paths). The black curve is the exact dynamic-programming boundary; the shaded region is the optimal exercise region. TDLSM-Net is the only method that qualitatively reproduces the monotone, smooth boundary across all rights levels.

targets. TDLSM-Net is the only method whose mean payoff *strictly increases* with data, reaching ≈46.5 at n=5,000 training paths.

At every rights level from ℓ=3 to ℓ=5, CQL and DQN place their exercise threshold at or above the strike K=100, effectively *never* exercising, which is reflected in their low payoffs. LSM-Net's boundary is erratic: it swings tens of dollars above and below the exact DP boundary, causing both premature exercise and missed opportunities. TDLSM-Net's boundary (orange) is smooth and systematically tracks below the exact boundary, meaning it exercises slightly later than optimal, but avoids the catastrophic over- or under-exercise of the other methods. Critically, this bias shrinks as rights increase (compare ℓ=5 with ℓ=3), because at higher rights levels there is more time remaining to amortize the approximation error across future exercises.

The mechanism is the same as in the tabular case, but its benefit is amplified by the deep horizon. TDLSM-Net replaces the noisy realized-path continuation with a bootstrapped target $\hat{y}_t = \gamma \hat{V}_{t+1}$, where $\hat{V}_{t+1}$ is already a fitted neural network, smooth by construction. Over 252 time steps and five rights layers, this reduction in target variance compounds: each of the $252 \times 5 = 1{,}260$ regressions receives a lower-variance training signal, so approximation errors do not cascade catastrophically through the backward sweep. By contrast, DQN and CQL must learn a single Q-function whose targets are bootstrapped from the same unstable Q-network, creating a circular dependency that stabilizes only with very large data budgets not available in the offline setting.